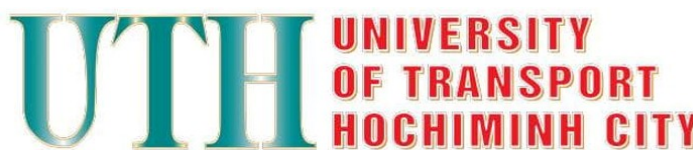


TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



KHÓA LUẬN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG HỎI ĐÁP PHÁP LUẬT TIẾNG VIỆT KẾT
HỢP RETRIEVAL-AUGMENTED GENERATION, ĐỒ THỊ TRI
THỨC VÀ CRITIC AGENT**

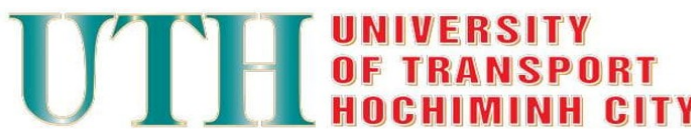
NGÀNH: CÔNG NGHỆ THÔNG TIN
CHUYÊN NGÀNH: CÔNG NGHỆ THÔNG TIN

NGƯỜI HƯỚNG DẪN
ThS. Huỳnh Thanh Việt

SINH VIÊN THỰC HIỆN:
Phan Quyết Tâm Phú; MSSV: 083205005445
Nguyễn Gia Huy; MSSV: 084205009123

TP. HỒ CHÍ MINH – Năm 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



PHAN QUYẾT TÂM PHÚ
NGUYỄN GIA HUY

XÂY DỰNG HỆ THỐNG HỎI ĐÁP PHÁP LUẬT TIẾNG VIỆT KẾT
HỢP RETRIEVAL-AUGMENTED GENERATION, ĐỒ THỊ TRI
THỨC VÀ CRITIC AGENT

NGÀNH: CÔNG NGHỆ THÔNG TIN
CHUYÊN NGÀNH: CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

NGƯỜI HƯỚNG DẪN

ThS. Huỳnh Thanh Việt
Viện Công nghệ thông tin và Điện, điện tử

TP. HỒ CHÍ MINH – Năm 2026



LỜI CAM ĐOAN

Chúng em tên là:

- Phan Quyết Tâm Phú - MSSV: 083205005445
- Nguyễn Gia Huy - MSSV: 084205009123.

Ngành: Công nghệ thông tin.

Chúng em xin cam đoan khóa luận tốt nghiệp với đề tài “Xây dựng hệ thống hỏi đáp pháp luật tiếng Việt kết hợp Retrieval-Augmented Generation, đồ thị tri thức và Critic Agent” là kết quả nghiên cứu, thiết kế và thực hiện của nhóm dưới sự hướng dẫn của ThS. Huỳnh Thanh Việt.

Toàn bộ số liệu và kết quả thực nghiệm được sử dụng trong khóa luận đều được tổng hợp từ các tệp bằng chứng của dự án, đồng thời được đối chiếu với mã nguồn, dữ liệu thực nghiệm và các tài liệu phương pháp có liên quan. Đối với những kết quả chưa có đầy đủ dữ liệu chi tiết để tái kiểm chứng, nhóm đã nêu rõ phạm vi xác minh và không đưa ra các kết luận vượt quá những bằng chứng hiện có.

Các tài liệu tham khảo được trích dẫn và trình bày theo chuẩn IEEE. Nhóm cam kết không sao chép trái phép các công trình, dữ liệu hoặc kết quả nghiên cứu của bất kỳ cá nhân hay tổ chức nào khác. Chúng em xin hoàn toàn chịu trách nhiệm về tính trung thực và nội dung của khóa luận này.

TP. Hồ Chí Minh, năm 2026

Sinh viên thực hiện

Phan Quyết Tâm Phú

Nguyễn Gia Huy

TÓM TẮT

Các hệ thống hỏi đáp pháp luật dựa trên Retrieval-Augmented Generation (RAG) thường truy xuất những đoạn văn bản có độ tương đồng cao với câu hỏi và cung cấp chúng làm ngữ cảnh cho mô hình ngôn ngữ lớn. Cách tiếp cận này giúp câu trả lời được xây dựng dựa trên nguồn dữ liệu bên ngoài, nhưng mức độ tương đồng ngữ nghĩa không đồng nghĩa với việc bảo toàn đầy đủ cấu trúc của quy phạm pháp luật. Nội dung của một quy định có thể được phân bố giữa phần dẫn nhập, nhiều Khoản và Điểm trong cùng một Điều; chế tài chính, chế tài bổ sung và biện pháp khắc phục hậu quả cũng có thể nằm ở các phần khác nhau. Ngoài ra, một Điều luật có thể dẫn chiếu đến các Điều khác có liên quan. Do đó, các child chunk thuộc nhóm top-k có thể phù hợp ở phạm vi cục bộ nhưng vẫn thiếu những căn cứ cần thiết để hình thành câu trả lời đầy đủ và chính xác. Trên cơ sở đó, khóa luận xây dựng và đánh giá một nguyên mẫu hỏi đáp pháp luật tiếng Việt kết hợp RAG, đồ thị tri thức và Critic Agent. Qua kiểm tra mã nguồn hiện hành, pipeline sử dụng 18 tệp DOCX để tạo 1.211 parent Article và 10.355 child chunk. Năm tệp PDF còn lại trong thư mục dữ liệu chưa được xử lý bởi bộ chunker. Chỉ mục Qdrant sử dụng mô hình mã hóa Base AITeamVN/Vietnamese_Embedding_v2, tạo vector dense 1.024 chiều, kết hợp với biểu diễn sparse BM25 và cơ chế hợp nhất thứ hạng Reciprocal Rank Fusion. Đồ thị Neo4j được sử dụng như một chỉ mục cấu trúc của văn bản pháp luật. Từ top-5 child chunk ban đầu, Critic Agent tạo câu trả lời nháp, phát hiện các bằng chứng còn thiếu trong cùng một Điều hoặc thông qua quan hệ THAM_CHIEU, đánh giá từng ứng viên bằng công cụ kiểm tra mức độ liên quan ngữ nghĩa và chỉ thực hiện sinh lại câu trả lời khi có bằng chứng bổ sung được chấp nhận. Hệ thống hiện được triển khai dưới dạng giao diện dòng lệnh và ứng dụng Chainlit. Đây là nguyên mẫu phục vụ nghiên cứu, chưa phải một hệ thống tư vấn pháp lý hoàn chỉnh.

Ba chế độ sử dụng chung Base Retriever được so sánh trên tập gồm 301 câu hỏi, bao gồm Naive RAG, Article Expansion và KG-based Critic. Theo bộ kết quả thực nghiệm tổng hợp, chỉ số Legal Completeness Rate của ba chế độ lần lượt đạt 43,77%, 72,72% và 81,33%. Số lượt gọi mô hình trực tuyến trung bình tương ứng là 1,0; 1,0 và 5,7 lượt cho mỗi câu hỏi. So với Article Expansion, KG-based Critic giảm 16,5% số token trong lần sinh câu trả lời cuối cùng và giảm 42,8% số ký tự của ngữ cảnh đầu vào. Tuy nhiên, phương pháp này phát sinh thêm chi phí do phải thực hiện các bước sinh câu trả lời nháp và đánh giá bằng chứng ứng viên. Các chỉ số LCR, RAGAS và mức tiêu thụ tài nguyên được giữ ở trạng thái số liệu tổng hợp theo báo cáo của phiên bản bài báo cuối cùng vì repository hiện chưa lưu đầy đủ raw output Critic cho 301 câu và raw scoring artifacts cần thiết để tái tính độc lập toàn bộ bảng. Số record của artefact cục bộ không được diễn giải thành phạm vi thực thi của toàn bộ thí nghiệm. Ngược lại, corpus, cấu trúc chỉ mục và các phép kiểm tra xác định của tập thử nghiệm đã được đối chiếu trực tiếp với mã nguồn hoặc các tệp bằng chứng tương ứng. Kết quả mô tả cho thấy việc khôi phục toàn bộ nội dung của một Điều giúp cải thiện độ bao phủ căn cứ pháp lý so với phương pháp chỉ sử dụng top-k child

chunk. Critic Agent tiếp tục mang lại cải thiện đối với nhóm câu hỏi cần bổ sung thông tin thông qua các quan hệ dẫn chiếu. Tuy nhiên, các kết luận của khóa luận chỉ có giá trị trong phạm vi snapshot dữ liệu, cấu hình hệ thống và quy trình đánh giá hiện có; kết quả của hệ thống không thay thế cho việc thẩm định và tư vấn của chuyên gia pháp luật.

MỤC LỤC

1. LỜI MỞ ĐẦU.....	1
1.1. Tính cấp thiết của đề tài.....	1
1.2. Tình hình nghiên cứu và phát biểu bài toán.....	2
1.3. Mục đích nghiên cứu.....	3
1.4. Nhiệm vụ và câu hỏi nghiên cứu.....	3
1.5. Phương pháp và phạm vi nghiên cứu.....	4
1.6. Các kết quả đạt được và đóng góp.....	4
1.7. Kết cấu khóa luận.....	5
2. CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN.....	6
2.1. Retrieval-Augmented Generation.....	6
2.2. Biểu diễn dense và cosine similarity.....	6
2.3. Truy xuất thừa bằng BM25.....	7
2.4. Reciprocal Rank Fusion.....	7
2.5. Chunking theo cấu trúc Điều - Khoản - Điểm.....	7
2.6. Đồ thị tri thức pháp luật.....	8
2.7. Critic Agent và bổ sung bằng chứng có chọn lọc.....	9
2.8. Nghiên cứu liên quan về legal QA và RAG.....	9
2.9. Chỉ tiêu đánh giá.....	10
2.9.1. Legal Completeness Rate.....	10
2.9.2. MRR và nDCG.....	11
2.9.3. Precision và recall theo Điều.....	11
2.9.4. Chỉ số tài nguyên.....	11
2.10. Khoảng trống nghiên cứu.....	12
3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	13
3.1. Yêu cầu chức năng.....	13
3.2. Yêu cầu phi chức năng.....	15
3.3. Kiến trúc tổng thể.....	15
3.4. Thiết kế dữ liệu văn bản.....	16

3.4.1. Tiền xử lý.....	17
3.4.2. Cơ chế parent/child.....	17
3.4.3. Kiểm soát trùng và va chạm ID.....	17
3.5. Thiết kế chỉ mục Qdrant.....	18
3.6. Thiết kế Knowledge Graph.....	21
3.6.1. Trích xuất hai lượt.....	21
3.6.2. Chuẩn hóa và placeholder.....	21
3.6.3. Lược đồ graph.....	21
3.7. Thiết kế ba chế độ RAG.....	22
3.7.1. Naive RAG.....	22
3.7.2. Article expansion.....	22
3.7.3. KG-based critic.....	22
3.8. Thuật toán Critic Agent.....	23
3.9. Thiết kế bộ đánh giá.....	27
3.9.1. Kiểm định xác định và đối chiếu toàn văn Điều.....	28
3.9.2. Chính sách diễn giải kết quả kiểm định.....	29
3.10. Thiết kế bảo mật và quản trị dữ liệu.....	30
4. CÀI ĐẶT VÀ THỰC NGHIỆM.....	31
4.1. Tổ chức mã nguồn.....	31
4.2. Môi trường và mô hình.....	32
4.3. Corpus và chỉ mục.....	33
4.4. Xây dựng và nạp đồ thị.....	34
4.5. Quy trình thao tác và các kịch bản đầu vào.....	35
4.5.4. Ứng viên không liên quan bị semantic gate loại.....	48
4.5.5. Gate exception và chính sách fail-open.....	49
4.5.6. Mixed-document safeguard sau khi mọi candidate bị từ chối.....	50
4.6. Quy trình chạy đánh giá.....	50
4.7. Chấm Legal Completeness.....	51
4.8. Kiểm thử phần mềm và kiểm soát lỗi.....	52

4.9. Khả năng tái lập.....	52
5. KẾT QUẢ VÀ THẢO LUẬN.....	53
5.1. Độ tin cậy của bộ testset.....	53
5.2. Kết quả truy xuất ban đầu của Base retriever.....	54
5.3. Legal Completeness Rate.....	56
5.4. Kết quả RAGAS.....	57
5.5. Precision, recall và độ dài ngữ cảnh theo Điều.....	58
5.6. Chi phí token và số lượt gọi mô hình.....	58
5.7. Trả lời các câu hỏi nghiên cứu.....	59
5.7.1. RQ1 - Khác biệt giữa ba mode.....	59
5.7.2. RQ2 - Ảnh hưởng của khôi phục và sửa bằng chứng.....	59
5.7.3. RQ3 - Đánh đổi chất lượng và chi phí.....	60
5.8. Phân tích lỗi và rủi ro.....	60
5.9. Giới hạn nghiên cứu.....	61
5.10. Hàm ý thiết kế.....	62
6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	63
6.1. Kết luận.....	63
6.2. Hướng phát triển.....	63
TÀI LIỆU THAM KHẢO.....	65

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Nội dung đầy đủ
AI	Artificial Intelligence — Trí tuệ nhân tạo
API	Application Programming Interface — Giao diện lập trình ứng dụng
BM25	Best Matching 25 — Thuật toán xếp hạng truy xuất thư
CLI	Command-Line Interface — Giao diện dòng lệnh
CUDA	Compute Unified Device Architecture
DOCX	Office Open XML Document
GPU	Graphics Processing Unit — Bộ xử lý đồ họa
ID	Identifier — Định danh
IEEE	Institute of Electrical and Electronics Engineers
JSONL	JSON Lines
KG	Knowledge Graph — Đồ thị tri thức
LCR	Legal Completeness Rate — Tỷ lệ đầy đủ pháp lý
LLM	Large Language Model — Mô hình ngôn ngữ lớn
MRR	Mean Reciprocal Rank
nDCG	normalized Discounted Cumulative Gain
NFKC	Normalization Form Compatibility Composition
NF4	NormalFloat 4-bit
OS	Operating System — Hệ điều hành

Từ viết tắt	Nội dung đầy đủ
QA	Question Answering — Hỏi đáp
RAG	Retrieval-Augmented Generation — Sinh tăng cường bằng truy xuất
RAGAS	Retrieval-Augmented Generation Assessment
RQ	Research Question — Câu hỏi nghiên cứu
RRF	Reciprocal Rank Fusion
SHA-256	Secure Hash Algorithm 256-bit
SLA	Service Level Agreement — Thỏa thuận mức dịch vụ
URL	Uniform Resource Locator — Địa chỉ tài nguyên thống nhất

DANH MỤC HÌNH ẢNH

Hình 3.1: Kiến trúc offline và online với Base hybrid retriever dùng chung cho ba chế độ.....	15
Hình 3.2: Vòng đời dữ liệu từ văn bản nguồn đến chỉ mục, đồ thị, testset và trả lời trực tuyến.....	16
Hình 3.3: Chuẩn hóa cấu trúc và biểu diễn truy xuất dense/BM25.....	18
Hình 3.4: Pipeline truy xuất lai dense/BM25, RRF và ba chế độ RAG.....	19
Hình 3.5: Lưu đồ quyết định của Critic: phát hiện thiếu, semantic gate và sinh lại có điều kiện.....	25
Hình 3.6: Quy trình kiểm định hai lớp cho testset và các tệp bằng chứng đầu ra cần lưu.....	28
Hình 4.1: Kết quả kiểm tra chỉ mục Qdrant Base của corpus thực nghiệm.....	34
Hình 4.2: Minh họa quan hệ THAM_CHIEU giữa các Điều trong Neo4j.....	35
Hình 4.3: Bốn kiểu khoảng trống bằng chứng và hành vi của Critic trong nguyên mẫu.....	36
Hình 4.4: Giao diện nguyên mẫu Chainlit khi chạy chế độ KG-based Critic.....	37
Hình 4.5: Kết quả so sánh ba chế độ trên cùng một câu hỏi bằng CLI.....	37
Hình 4.6: Trace quyết định của Critic Agent từ kết quả chạy thực tế.....	38
Hình 4.7: Case study không phát hiện khoảng trống bằng chứng.....	42
Hình 4.8: Case study bổ sung bằng chứng trong cùng Điều.....	45
Hình 4.9: Case study bổ sung Điều qua quan hệ THAM_CHIEU.....	48
Hình 5.1: Legal Completeness Rate của ba chế độ dùng chung Base retriever.....	56
Hình 5.2: Đánh đổi giữa LCR, token của lần sinh cuối, số ký tự ngữ cảnh và số lượt gọi mô hình online.....	58

DANH MỤC BẢNG

Bảng 2.1. Cấu trúc định danh chunk và vai trò trong pipeline.....	8
Bảng 3.1. Yêu cầu người dùng và tiêu chí chấp nhận.....	14
Bảng 3.2. Các bước xử lý một truy vấn từ đầu vào đến ngữ cảnh sinh.....	19
Bảng 3.3. Payload tối thiểu của child và parent chunk.....	20
Bảng 3.4. Các nhóm nút và quan hệ chính trong đồ thị.....	21
Bảng 3.5. So sánh vận hành của ba chế độ.....	23
Bảng 3.6. Bảng quyết định các nhánh vận hành đã xác minh và đề xuất.....	26
Bảng 3.7. Phân bố bộ câu hỏi đánh giá.....	27
Bảng 3.8. Chín kiểm tra dùng để kiểm định bộ testset.....	29
Bảng 4.1. Các module chính của hệ thống.....	31
Bảng 4.2. Cấu hình thực nghiệm chính.....	32
Bảng 4.3. Các kịch bản nhập câu hỏi và hành vi mong đợi.....	38
Bảng 4.4. Case study không phát hiện khoảng trống bằng chứng (cat4_02).....	40
Bảng 4.5. Case study bổ sung bằng chứng trong cùng Điều (cat1_01).....	42
Bảng 4.6. Case study bổ sung Điều qua quan hệ THAM_CHIEU (cat2_07).....	45
Bảng 5.1. Kết quả kiểm định testset do tài liệu phương pháp báo cáo.....	53
Bảng 5.2. Chỉ số truy xuất Base theo hai phiên bản dữ liệu bằng chứng.....	54
Bảng 5.3. Legal Completeness Rate trên 301 câu hỏi.....	56
Bảng 5.4. Kết quả RAGAS của cấu hình Base.....	57
Bảng 5.5. Chỉ số context ở cấp Điều.....	58
Bảng 5.6. Chỉ số chi phí online trung bình trên mỗi câu hỏi.....	59

1. LỜI MỞ ĐẦU

1.1. Tính cấp thiết của đề tài

Văn bản pháp luật là nguồn tri thức có tính cấu trúc cao, thay đổi theo thời gian và đòi hỏi người đọc đồng thời xác định đúng văn bản, đúng Điều, đúng Khoản, đúng Điểm, phạm vi áp dụng và các dẫn chiếu liên quan. Khi người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên, từ khóa trong câu hỏi có thể khác với cách diễn đạt trong văn bản. Một hệ thống tra cứu chỉ dựa trên tìm kiếm từ khóa vì vậy có thể bỏ sót quy định dùng từ đồng nghĩa hoặc mô tả pháp lý khác. Ngược lại, một mô hình ngôn ngữ lớn có thể tạo câu trả lời trôi chảy nhưng không có cơ chế bảo đảm mọi kết luận đều dựa trên văn bản hiện hành.

RAG kết hợp bộ truy xuất với mô hình sinh, cho phép mô hình điều kiện hóa câu trả lời trên các tài liệu lấy từ kho dữ liệu ngoài [1]. Kho tri thức có thể cập nhật độc lập với tham số của mô hình, đồng thời các đoạn ngữ cảnh được truy xuất tạo cơ sở để kiểm tra câu trả lời. Đối với tiếng Việt, bài toán hỏi đáp pháp luật đã được nghiên cứu ở mức truy xuất Điều luật, trong đó câu hỏi được ánh xạ tới các Điều liên quan [2]. Các hệ thống pháp lý thực tế cũng đã kết hợp tìm kiếm thưa, biểu diễn dense và reranking [3]. Tuy nhiên, độ liên quan của từng đoạn chưa đồng nghĩa với độ đầy đủ của căn cứ pháp lý.

Trong dữ liệu pháp luật Việt Nam, một Điều thường có lời dẫn và nhiều Khoản; mỗi Khoản có thể tiếp tục chứa nhiều Điểm. Quy định về một hành vi có thể đặt mức phạt tiền ở một Khoản, hình thức xử phạt bổ sung ở Khoản khác và biện pháp khắc phục hậu quả ở phần cuối Điều. Một quy tắc còn có thể dẫn chiếu sang Điều khác bằng các cụm như “theo quy định tại Điều...”. Nếu chỉ lấy top-k chunk riêng lẻ, hệ thống có thể trả lời đúng phần mức phạt nhưng bỏ sót tịch thu phương tiện, đình chỉ hoạt động hoặc nghĩa vụ khắc phục. Nếu luôn mở rộng toàn bộ nhiều Điều, ngữ cảnh tăng nhanh và có nguy cơ chứa chi tiết không liên quan; nghiên cứu về ngữ cảnh dài cho thấy khả năng sử dụng bằng chứng của mô hình còn phụ thuộc vị trí của bằng chứng trong chuỗi đầu vào [4].

Đồ thị tri thức cung cấp một cách biểu diễn khác: thực thể pháp lý trở thành nút, còn quan hệ cấu trúc và dẫn chiếu trở thành cạnh. Các hệ thống truy xuất dựa trên đồ thị có khả năng kết nối bằng chứng nằm ở nhiều đoạn [5]. Từ quan sát đó, khóa luận đặt Critic Agent sau bước sinh nháp. Critic không thay thế retriever mà dùng cấu trúc đồ thị để trả lời câu hỏi kiểm tra: các Điều đang xét còn thiếu phần nào, có chế tài kếp hay không, có dẫn chiếu ra Điều khác hay không, và phần được phát hiện có thật sự liên quan đến câu hỏi hay không.

1.2. Tình hình nghiên cứu và phát biểu bài toán

Các nghiên cứu liên quan đã khảo sát ba hướng chính: truy xuất Điều luật cho câu hỏi pháp lý tiếng Việt [2], hỏi đáp pháp lý kết hợp truy xuất thưa, biểu diễn ngữ nghĩa và xếp hạng lại [3], cùng các hệ thống RAG/đồ thị và bộ đánh giá như HippoRAG, LegalBench, LawBench, KoBLEX, CLERC, RAGAS và ARES [5], [8]–[13]. Các công trình này khẳng định vai trò của truy xuất và đánh giá đa chiều, nhưng chưa trực tiếp kiểm tra cơ chế khôi phục cấu trúc Điều–Khoản–Điểm rồi bổ sung có chọn lọc bằng quan hệ pháp lý trong phạm vi corpus của đề tài.

Đầu vào của hệ thống là một câu hỏi pháp luật tiếng Việt và một kho văn bản đã được chuẩn hóa. Đầu ra mong muốn là câu trả lời có căn cứ, đúng trọng tâm và bao phủ các ý pháp lý bắt buộc. Bài toán không chỉ là tìm đoạn giống câu hỏi nhất. Hệ thống cần giải quyết đồng thời bốn yêu cầu:

1. Truy xuất đúng đoạn nhỏ để tránh đưa quá nhiều nội dung không liên quan vào prompt.
2. Khôi phục cấu trúc toàn Điều khi câu trả lời cần nhiều Khoản hoặc Điểm trong cùng Điều.
3. Lăn theo dẫn chiếu để tìm Điều liên quan nằm ngoài kết quả top-k ban đầu.
4. Kiểm soát chi phí bằng cách chỉ bổ sung và sinh lại khi phần mới thực sự hữu ích.

Khóa luận xem “đầy đủ pháp lý” là khả năng câu trả lời thể hiện đúng các required facts đã gắn với từng câu hỏi đánh giá. Đây là tiêu chí vận hành cho thí nghiệm, không phải kết luận về giá trị tư vấn pháp lý của hệ thống. Bộ required facts và đáp án tham chiếu chưa được chuyên gia luật xác nhận độc lập, vì vậy được xem là silver standard.

1.3. Mục đích nghiên cứu

Mục tiêu tổng quát là xây dựng và đánh giá kiến trúc RAG nhận biết cấu trúc cho hỏi đáp pháp luật tiếng Việt. Các mục tiêu cụ thể gồm:

- Xây dựng bộ tách văn bản theo Điều - Khoản - Điểm và cơ chế parent/child chunk.
- Tạo chỉ mục truy xuất lai kết hợp embedding tiếng Việt và BM25.
- Trích xuất đồ thị tri thức pháp luật, đặc biệt là quan hệ cấu trúc, chế tài và THAM_CHIEU.
- Thiết kế Critic Agent phát hiện bằng chứng thiếu, lọc ngữ nghĩa và sinh lại có điều kiện.
- So sánh công bằng ba chế độ xử lý trên cùng truy vấn, corpus và Base retriever dùng chung.
- Đánh giá cả chất lượng câu trả lời, chất lượng truy xuất và chi phí tài nguyên.

1.4. Nhiệm vụ và câu hỏi nghiên cứu

Khóa luận trả lời ba câu hỏi nghiên cứu:

- **RQ1:** Naive RAG, mở rộng toàn Điều và KG-based critic khác nhau như thế nào về Legal Completeness Rate, đặc biệt trên câu hỏi đa cấu trúc và tham chiếu chéo?
- **RQ2:** Việc khôi phục toàn Điều và bổ sung bằng chứng theo cấu trúc ảnh hưởng như thế nào đến Article precision, Article recall, RAGAS và độ dài context trên Base retriever?

- RQ3: Mức cải thiện LCR của Critic Agent đi kèm sự đánh đổi nào về số lượt gọi mô hình online, token của lần sinh câu trả lời cuối và số ký tự ngữ cảnh?

1.5. Phương pháp và phạm vi nghiên cứu

Khóa luận sử dụng phương pháp phân tích tài liệu và mã nguồn, thực nghiệm so sánh có kiểm soát trên cùng Base hybrid retriever, đối chiếu định lượng bằng MRR, nDCG, precision/recall theo Điều, RAGAS và Legal Completeness Rate, đồng thời phân tích trace theo các kịch bản thiếu bằng chứng. Python được dùng để xử lý dữ liệu và tổng hợp chỉ số; Qdrant lưu chỉ mục dense/BM25 và Neo4j lưu đồ thị quan hệ pháp lý.

Corpus của dự án gồm 23 văn bản pháp luật Việt Nam, trong đó 18 tệp DOCX đi qua quy trình chunking và năm tệp PDF được lưu làm nguồn nhưng chưa được phân tích vào chỉ mục thí nghiệm. Nội dung tập trung vào sở hữu trí tuệ, công nghệ thông tin, an ninh mạng, dữ liệu cá nhân, giao dịch điện tử, viễn thông, thương mại điện tử và xử phạt hành chính có liên quan. Kết quả không đại diện cho toàn bộ hệ thống pháp luật Việt Nam.

Khóa luận tập trung vào quy trình dữ liệu, Base hybrid retrieval, đồ thị, Critic Agent, đánh giá ngoại tuyến và nguyên mẫu CLI/Chainlit. Tư vấn pháp lý chính thức, vận hành thực tế, bảo đảm SLA và kiểm định bởi luật sư không thuộc phạm vi kết luận. Các thử nghiệm fine-tuned và GTE chỉ được nhắc như lịch sử nghiên cứu: dự án không có checkpoint fine-tuned, còn collection mang tên GTE có kích thước 1.024 không khớp cấu hình chính thức 768 chiều, nên cả hai không đủ điều kiện làm trực kết quả có thể tái lập.

1.6. Các kết quả đạt được và đóng góp

Đóng góp thứ nhất là quy trình chunking bảo toàn cấu trúc pháp luật. Mỗi phần nhỏ nhất có thể truy xuất được gán định danh vị trí pháp lý, đồng thời các child chunk cùng Điều được liên kết với parent chunk toàn Điều. Cách tổ chức này tách hai mục tiêu thường xung đột: độ chính xác của truy xuất và độ đầy đủ của ngữ cảnh.

Đóng góp thứ hai là đồ thị tri thức được xây dựng bằng chiến lược trích xuất hai lượt và hậu xử lý rule-based. Đồ thị không chỉ lưu Điều, Khoản, Điểm mà còn biểu diễn hành vi, chế tài và cạnh THAM_CHIEU. Placeholder node giúp ghi nhận một đích dẫn chiếu chưa có trong đồ thị thay vì làm mất quan hệ.

Đóng góp thứ ba là Critic Agent bổ sung bằng chứng theo nhu cầu. Thay vì luôn mở rộng mọi Điều, Critic phát hiện ba nhóm thiếu sót: chế tài kép trong cùng Điều, thiếu thành phần cấu trúc của Điều, và dẫn chiếu chéo giữa các Điều. Ứng viên còn phải qua cổng liên quan ngữ nghĩa trước khi đi vào ngữ cảnh sinh cuối.

Đóng góp thứ tư là khung đánh giá ba mode trên cùng Base retriever, tách chất lượng retrieval, chất lượng answer và chi phí. Bên cạnh RAGAS và các metric xếp hạng, khóa luận sử dụng Legal Completeness Rate để đo mức bao phủ required facts đặc thù pháp lý. Bộ testset được mô tả và kiểm định theo hai lớp gồm quy tắc xác định và sáu tiêu chí LLM-as-a-judge; các trường hợp bị gán cờ được công khai để rà soát. Các số liệu chưa thể tái tạo từ output hiện có được gắn nhãn “nguồn báo cáo”, tránh trộn với kết quả đã tái tính.

1.7. Kết cấu khóa luận

Chương 2 trình bày cơ sở lý thuyết và nghiên cứu liên quan. Chương 3 phân tích yêu cầu, dữ liệu và thiết kế kiến trúc. Chương 4 mô tả cài đặt, cấu hình và quy trình thực nghiệm. Chương 5 báo cáo kết quả, trả lời các câu hỏi nghiên cứu và phân tích giới hạn. Chương 6 tổng kết đóng góp và đề xuất hướng phát triển.

2. CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN

2.1. Retrieval-Augmented Generation

RAG gồm hai thành phần chính: retriever xác định tập tài liệu có khả năng liên quan và generator sinh chuỗi đầu ra dựa trên câu hỏi cùng tài liệu truy xuất [1]. Với truy vấn x , chuỗi trả lời y và tài liệu ẩn z , xác suất sinh có thể được mô tả bằng phép lấy tổng trên các tài liệu ứng viên:

$$p(y/x) = \sum_{z \in Z} p_{\eta}(z/x) p_{\theta}(y/x, z) \quad (2.1)$$

Trong đó, $p_{\eta}(z/x)$ là phân phối của retriever và $p_{\theta}(y/x, z)$ là xác suất của generator. Công thức cho thấy chất lượng đầu ra phụ thuộc đồng thời vào khả năng lấy đúng bằng chứng và khả năng sử dụng bằng chứng của mô hình sinh. Nếu tài liệu cần thiết không xuất hiện trong tập truy xuất, generator không thể tự động khôi phục căn cứ chỉ bằng prompt. Nếu tập truy xuất quá rộng, thông tin đúng có thể bị pha loãng bởi nội dung ít liên quan.

Trong khóa luận, RAG được dùng theo nghĩa kiến trúc hệ thống chứ không sao chép nguyên cấu hình huấn luyện của Lewis và cộng sự. Retriever là hệ thống hybrid Qdrant; generator là Qwen2.5-7B-Instruct chạy qua giao diện tương thích OpenAI/Ollama; kho tri thức là corpus văn bản pháp luật Việt Nam.

2.2. Biểu diễn dense và cosine similarity

Dense retrieval ánh xạ câu hỏi và đoạn văn thành vector liên tục. Các vector gần nhau biểu thị nội dung có quan hệ ngữ nghĩa. Với vector truy vấn q và vector chunk d , cosine similarity được tính như sau [6]:

$$\text{sim}(q, d) = \frac{q^T d}{\|q\|_2 \|d\|_2} \quad (2.2)$$

Trong cấu hình chính đã xác minh, embedding được chuẩn hóa L2, Qdrant dùng khoảng cách cosine và mô hình dense là AITeamVN/Vietnamese_Embedding_v2 với vector 1.024 chiều [16]. Hai biến thể fine-tuned và GTE từng xuất hiện trong các bản thảo cũ không được dùng làm trục của khóa luận: dự án không có checkpoint fine-

tuned, trong khi collection mang tên GTE vẫn khai báo và lưu vector 1.024 chiều dù cấu hình chính thức của gte-multilingual-base cho biết đầu ra 768 chiều [7], [15]. Vì vậy, số liệu lịch sử của hai biến thể này không được diễn giải như kết quả có thể tái lập.

2.3. Truy xuất thừa bằng BM25

Dense retrieval phù hợp với tương đồng ngữ nghĩa, còn BM25 hữu ích khi câu hỏi chứa số Điều, ký hiệu văn bản, thuật ngữ hoặc cụm từ pháp lý chính xác. Với từ t , tài liệu d , tần suất $f(t, d)$, độ dài $|d|$, độ dài trung bình $avgdl$ và các tham số k_1 , b , điểm BM25 có dạng [6]:

$$BM25(q, d) = \sum_{t \in q} IDF(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{avgdl}\right)} \quad (2.3)$$

Module BM25SparseVectorizer dùng token chữ thường sau chuẩn hóa Unicode NFKC, $k_1 = 1,5$, $b = 0,75$ và IDF làm tròn. Vector query giữ các term phù hợp để tích vô hướng sparse trong Qdrant tương ứng với điểm BM25 đã lưu ở phía corpus.

2.4. Reciprocal Rank Fusion

Kết quả dense và sparse có thang điểm khác nhau. Reciprocal Rank Fusion (RRF) kết hợp thứ hạng thay vì cộng trực tiếp điểm. Với tài liệu d , tập danh sách xếp hạng L và hằng số k_{RRF} , điểm hợp nhất là:

$$RRF(d) = \sum_{L \in L} \frac{1}{k_{RRF} + rank_L(d)} \quad (2.4)$$

Trong pipeline, Qdrant chạy dense search và BM25 sparse search song song trên collection child chunk, sau đó thực hiện fusion trước khi trả về top-k. Cách kết hợp này cho phép một chunk đạt thứ hạng cao nhờ tương đồng ngữ nghĩa, khớp thuật ngữ pháp lý, hoặc cả hai.

2.5. Chunking theo cấu trúc Điều - Khoản - Điểm

Chunking theo số token cố định có thể cắt giữa một Khoản hoặc tách điều kiện khỏi hệ quả pháp lý. Bộ VBPLChunker dùng biểu thức chính quy để nhận diện tiêu

đề Điều, Khoản đánh số và Điểm ký hiệu chữ cái. Child chunk có thể là lời dẫn Điều, nội dung Khoản hoặc nội dung Điểm. Khi một child quá dài, hệ thống tách tiếp theo đoạn, câu và cuối cùng theo từ, mặc định không chồng lấn.

Mỗi child được gán `diem_id`, `parent_id`, loại cấu trúc, vị trí và tiền tố pháp lý. Định danh tổng quát là `<doc_id>_D<article>`; Khoản và Điểm thêm hậu tố `_K<clause>` và `_P<point>`; phần bị tách thêm `_S<n>`. Các child cùng `diem_id` được nối theo thứ tự nguồn thành parent chunk đại diện toàn văn Điều. Cấu trúc hai cấp cho phép mode naive dùng các phần nhỏ, còn mode `article_expand` và `critic` lấy toàn văn Điều khi cần.

Bảng 2.1. Cấu trúc định danh chunk và vai trò trong pipeline

Thành phần	Ví dụ định danh	Nội dung	Vai trò
Điều	..._D98	Nút cấu trúc của Điều 98	Liên kết parent và graph
Khoản	..._D98_K1	Một Khoản độc lập	Child retrieval
Điểm	..._D98_K1_Pa	Điểm a của Khoản 1	Child retrieval chi tiết
Parent	..._D98_PARENT	Toàn văn Điều 98	Mở rộng ngữ cảnh
Split	..._S2	Phần thứ hai của child dài	Giữ giới hạn đầu vào embedding

2.6. Đồ thị tri thức pháp luật

Một đồ thị tri thức có thể biểu diễn bằng bộ ba:

$$G=(V,E,R) \quad (2.5)$$

Trong đó, V là tập nút thực thể, E là tập cạnh và R là tập loại quan hệ. Trong đề tài, các loại nút chính gồm Điều, Khoản, Điểm, HànhVi, ChếTai, ChuThe, NghĩaVu, QuyenHan, KhaiNiem và DieuKien. Quan hệ cấu trúc gồm CO_KHOAN,

CO_DIEM; quan hệ ngữ nghĩa gồm QUY_DINH_HANH_VI, CHE_TAI_CHINH, CHE_TAI_BO_SUNG và THAM_CHIEU.

Đồ thị không được dùng như một nguồn thay thế văn bản gốc. Nó là chỉ mục cấu trúc giúp Critic phát hiện nơi có khả năng thiếu. Nội dung cuối cùng đưa vào generator vẫn là parent Article lấy từ Qdrant. Ranh giới này giảm rủi ro biến output trích xuất của LLM thành căn cứ pháp lý không thể kiểm tra.

2.7. Critic Agent và bổ sung bằng chứng có chọn lọc

Critic Agent hoạt động sau lần sinh nháp. Trước hết, các Điều trong top-k được lọc bằng tỷ lệ so với điểm Điều cao nhất và giới hạn số Điều cần kiểm tra. Critic truy vấn Neo4j để tìm ba tín hiệu:

- Một hành vi trong Điều có đồng thời CHE_TAI_CHINH và CHE_TAI_BO_SUNG.
- Số thành phần Khoản/Điều đã truy xuất nhỏ hơn tổng số thành phần của Điều.
- Có cạnh THAM_CHIEU đi ra hoặc đi vào giữa Điều đang xét và Điều chưa có trong ngữ cảnh.

Với mỗi ứng viên, cổng liên quan ngữ nghĩa yêu cầu mô hình trả lời nhị phân yes/no đối với câu hỏi và đoạn ứng viên. Chỉ ứng viên được chấp nhận mới được đưa vào phần ngữ cảnh bổ sung; exception hiện được xử lý fail-open và phải được ghi riêng, không được xem là một quyết định semantic yes. Nếu có bằng chứng bổ sung, hệ thống sinh lại từ đầu bằng prompt chung. Khi không có ứng viên được chấp nhận, safeguard có thể lập parent Article đứng đầu nếu top-k trộn từ ít nhất hai văn bản; chỉ khi cả safeguard không kích hoạt thì hệ thống giữ câu trả lời nháp. Cơ chế này tách hai câu hỏi: cấu trúc cho biết “có thể thiếu ở đâu”, còn semantic gate quyết định “phần đó có cần cho câu hỏi hiện tại hay không”.

2.8. Nghiên cứu liên quan về legal QA và RAG

Kien và cộng sự xây dựng mô hình truy xuất Điều luật cho câu hỏi pháp lý tiếng Việt trên 5.922 câu hỏi, sử dụng biểu diễn neural attentive và đánh giá bằng recall,

nDCG [2]. Nghiên cứu này chứng minh tính khả thi của Article retrieval nhưng không xử lý trực tiếp sinh câu trả lời hoặc phát hiện căn cứ thiếu sau truy xuất. Khazaeli và cộng sự kết hợp sparse search, embedding và reranking trong hệ thống legal QA định dạng tự do [3]. Đây là tiền đề gần với hybrid retrieval của khóa luận.

LegalBench và LawBench cho thấy năng lực pháp lý của mô hình cần được đánh giá trên nhiều nhóm tác vụ, không thể rút gọn thành một điểm duy nhất [8], [9]. KoBLEX đánh giá hỏi đáp pháp luật có hỗ trợ điều khoản và suy luận nhiều bước [10]. CLERC liên kết truy xuất án lệ với sinh phân tích tăng cường truy xuất [11]. Các bộ dữ liệu này khác hệ thống pháp luật và ngôn ngữ của đề tài, nhưng củng cố nhu cầu tách retrieval, evidence grounding và reasoning.

HippoRAG dùng đồ thị để hỗ trợ truy xuất dài hạn và liên kết thông tin qua nhiều đoạn [5]. Khóa luận chia sẻ trực giác rằng quan hệ giữa các đoạn có thể bổ sung cho similarity search, nhưng graph ở đây được thiết kế theo cấu trúc văn bản pháp luật Việt Nam và phục vụ missing-evidence critique, không nhằm tái hiện toàn bộ HippoRAG.

RAGAS tách các chiều context precision, faithfulness, answer relevancy và answer correctness [12]. ARES cũng xem xét context relevance, answer faithfulness và answer relevance [13]. Hai hướng đánh giá nhấn mạnh rằng một hệ thống RAG có thể truy xuất tốt nhưng sinh chưa faithful, hoặc sinh trôi chảy nhưng context chưa chính xác. Đề tài bổ sung LCR để đo mức bao phủ các facts pháp lý bắt buộc.

2.9. Chỉ tiêu đánh giá

2.9.1. Legal Completeness Rate

Với câu hỏi i , gọi F_i là tập required facts và $J(a_i, f)$ là phán đoán nhị phân của judge đối với câu trả lời a_i và fact f . Độ bao phủ câu hỏi là:

$$C_i = \frac{1}{|F_i|} \sum_{f \in F_i} 1[J(a_i, f) = 1] \quad (2.6)$$

LCR trên N câu hỏi là:

$$LCR = \frac{100}{N} \sum_{i=1}^N C_i \quad (2.7)$$

LCR thuộc khoảng $[0, 100]\%$. Judge dùng GPT-4o-mini ở nhiệt độ 0, tách biệt với generator Qwen. Prompt chấp nhận cách diễn đạt tương đương nhưng yêu cầu đúng con số và chi tiết pháp lý. Do LLM judge có thể chịu thiên lệch vị trí, độ dài và xu hướng tự ưu tiên [14], LCR cần được diễn giải cùng giới hạn chưa có validation với chuyên gia.

2.9.2. *MRR và nDCG*

Với hạng của kết quả liên quan đầu tiên r_i , truncated MRR@K là:

$$MRR@K = \frac{1}{N} \sum_{i=1}^N \frac{1[r_i \leq K]}{r_i} \quad (2.8)$$

nDCG@5 đánh giá chất lượng thứ hạng trong năm vị trí đầu:

$$nDCG@5 = \frac{1}{N} \sum_{i=1}^N \frac{\sum_{r=1}^5 (2^{rel_r} - 1) / \log_2(r+1)}{IDCG_i@5} \quad (2.9)$$

Hai chỉ số thuộc $[0, 1]$; giá trị cao hơn tương ứng kết quả liên quan xuất hiện sớm và thứ tự top-5 tốt hơn [6].

2.9.3. *Precision và recall theo Điều*

Với tập Điều trong ngữ cảnh cuối A_i và tập Điều tham chiếu G_i :

$$P_i = \frac{|A_i \cap G_i|}{|A_i|}, R_i = \frac{|A_i \cap G_i|}{|G_i|} \quad (2.10)$$

Precision cho biết tỷ lệ Điều đã lấy là đúng, còn recall cho biết tỷ lệ Điều đúng đã được bao phủ. Metric này khác Context Precision của RAGAS: một metric hoạt động ở cấp tập ID Điều, metric còn lại đánh giá mức tập trung của các context đối với câu hỏi.

2.9.4. *Chỉ số tài nguyên*

Average Context Characters là số ký tự trung bình của bằng chứng đưa vào lần sinh cuối. Final-call Tokens per Question là tổng prompt token và completion token

do provider báo cáo cho chính lượt tạo câu trả lời cuối; Online LLM Calls per Question đếm các lượt sinh nháp, gate và sinh cuối trong luồng được đánh giá. LCR và RAGAS là bộ chấm ngoại tuyến nên không được cộng vào chi phí online. Tổng pipeline token từ nguồn cũ không được dùng trong so sánh chính vì baseline cũ có tính router trong khi script đánh giá hiện hành đặt skip_router=True; trộn hai ranh giới đo sẽ làm sai diễn giải chi phí.

2.10. Khoảng trống nghiên cứu

Các nghiên cứu trước cung cấp nền tảng về RAG, truy xuất pháp luật, hỏi đáp nhiều bước và đánh giá tự động, nhưng chưa có một phương pháp trực tiếp trong phạm vi đề tài để so sánh ba chiến lược: giữ top-k chunk, mở rộng toàn Điều, và phát hiện thiếu bằng đồ thị rồi bổ sung có chọn lọc. Khoảng trống chính của đề tài là cơ chế kiểm tra sau truy xuất dựa trên cấu trúc pháp lý, nhằm cải thiện tính đầy đủ mà không mặc định tăng toàn bộ ngữ cảnh.

3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Yêu cầu chức năng

Hệ thống được thiết kế cho hai luồng chính: xây dựng dữ liệu offline và trả lời câu hỏi online. Luồng offline phải đọc văn bản DOCX, tách cấu trúc, tạo embedding, xây chỉ mục sparse/dense, trích xuất graph và nạp dữ liệu vào Qdrant/Neo4j. Luồng online phải nhận câu hỏi, truy xuất, chạy một trong ba mode, sinh câu trả lời và ghi lại các tệp bằng chứng cần cho đánh giá.

Các yêu cầu chức năng cụ thể gồm:

- Nhận dạng Điều, Khoản, Điểm và bảo toàn thứ tự nguồn.
- Tạo parent/child chunks với định danh ổn định.
- Hỗ trợ dense search, sparse search và RRF.
- Ánh xạ child hit về parent Điều.
- Truy vấn graph cho chế tài kép, thiếu cấu trúc và tham chiếu hai chiều.
- Lọc candidate theo điểm và semantic relevance.
- Chạy độc lập mode naive, article_expand, critic trên cùng input và Base retriever.
- Trả hoặc lưu answer, context, draft, critic report, token và số lần gọi LLM theo khả năng của từng entry point.
- Hỗ trợ resume khi đánh giá theo batch bị gián đoạn.

Người dùng mục tiêu ở phạm vi nguyên mẫu gồm: người tra cứu đặt câu hỏi pháp luật bằng tiếng Việt; người nghiên cứu so sánh ba mode; người quản trị cập nhật corpus, chỉ mục và đồ thị. Người tra cứu không phải biết cú pháp Điều/Khoản/Điểm, nhưng câu hỏi nên mô tả đủ chủ thể, hành vi, điều kiện và thông tin cần biết. Router được dùng trong phiên chat để tách hội thoại thông thường khỏi câu hỏi pháp luật; script đánh giá đặt skip_router=True để ba mode được chấm trên cùng câu hỏi và không cộng chi phí router vào số lượt gọi.

Bảng 3.1. Yêu cầu người dùng và tiêu chí chấp nhận

Mã	Vai trò/nhu cầu	Hành vi hệ thống	Tiêu chí chấp nhận
UR-01	Người tra cứu hỏi bằng tiếng Việt tự nhiên	Router phân loại legal hoặc chit_chat	Câu pháp luật đi vào retrieval; câu chào hỏi không truy xuất corpus
UR-02	Cần câu trả lời có căn cứ	Trả lời từ context đã lấy, kèm Điều/Khoản/Điểm khi có	Không viện dẫn ngoài context; cảnh báo nếu bằng chứng không đủ
UR-03	Câu hỏi có nhiều chế tài/cấu trúc	Cho phép chọn article_expand hoặc critic	Có thể khôi phục parent Điều và ghi danh sách Điều đã dùng
UR-04	Câu hỏi có dẫn chiếu	Critic kiểm tra cạnh THAM_CHIEU và semantic gate	Candidate vào final context khi gate trả yes; exception hiện hành là fail-open và phải được ghi nhận như rủi ro
UR-05	Người nghiên cứu cần so sánh	Chạy naive, article_expand, critic trên cùng query	--compare-all tạo ba kết quả có cùng đầu vào và cấu hình retrieval
UR-06	Người quản trị cập nhật dữ liệu	Re-ingest Qdrant và xây lại Neo4j theo cùng snapshot	Collection, graph, model dimension và corpus version khớp nhau

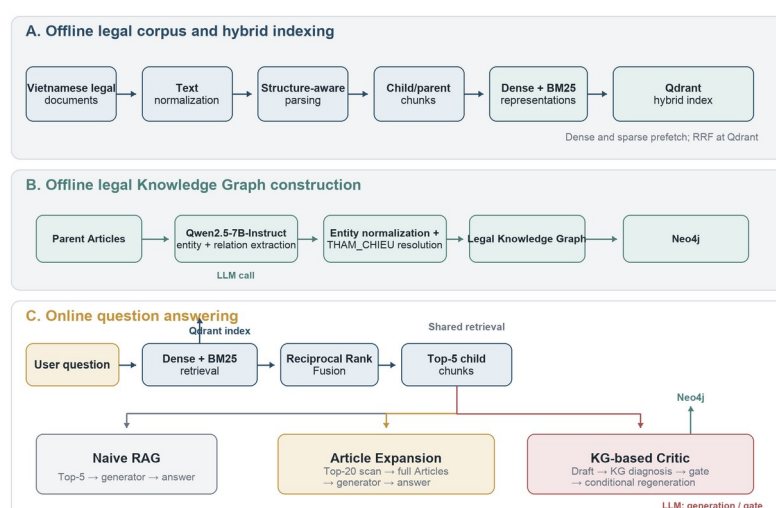
Mã	Vai trò/nhu cầu	Hành vi hệ thống	Tiêu chí chấp nhận
UR-07	Cần truy vết thực nghiệm	Lưu context, draft, final, critic report và token	Script đánh giá có schema lưu theo record; UI hiện chưa hiển thị mọi trường trung gian

3.2. Yêu cầu phi chức năng

Tính truy vết được ưu tiên: mỗi chunk cần chứa vị trí pháp lý và ID nguồn; câu trả lời cần có context để kiểm tra. Tính tái lập yêu cầu lưu cấu hình, checkpoint, hash dữ liệu và output theo từng câu hỏi. Tính an toàn yêu cầu khóa truy cập Qdrant, Neo4j và API không nằm trong mã nguồn; .gitignore loại .env, API key và dữ liệu index khỏi commit.

Hệ thống chưa phải công cụ tư vấn pháp lý. Giao diện phải có cảnh báo phạm vi, ngày hiệu lực và yêu cầu đối chiếu văn bản gốc. Dữ liệu luật thay đổi theo thời gian nên mỗi văn bản cần metadata hiệu lực, nguồn phát hành, ngày thu thập và phiên bản hợp nhất.

3.3. Kiến trúc tổng thể

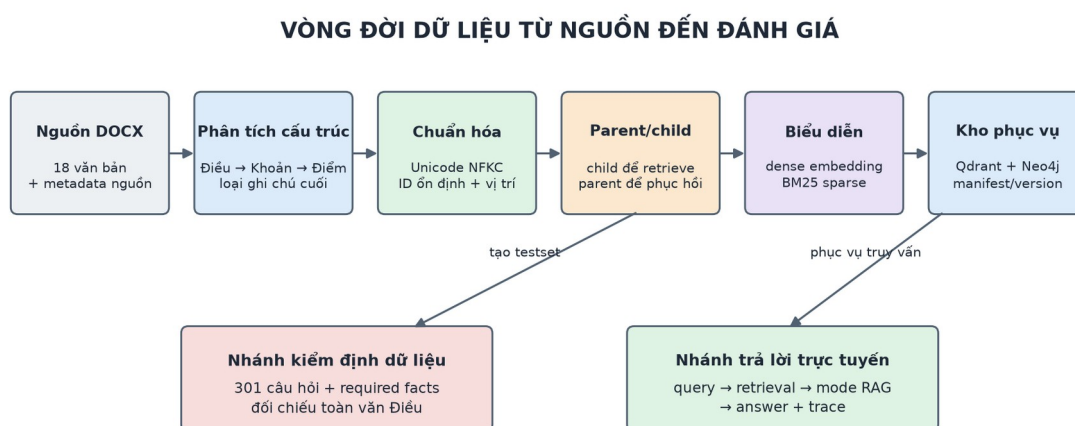


Hình 3.1: Kiến trúc offline và online với Base hybrid retriever dùng chung cho ba chế độ

Kiến trúc gồm hai phần. Ở phía offline, VBPLChunker tạo child/parent chunks; embedding và BM25 tạo hai loại vector để nạp Qdrant; Qwen2.5-7B-Instruct thực hiện trích xuất entity/relation, sau đó graph builder chuẩn hóa và Neo4j ingestor ghi đồ thị. Ở phía online, một truy vấn đi qua hybrid retrieval dùng chung. Từ đây pipeline rẽ sang ba mode. Naive sinh trực tiếp từ top-5 child. Article expansion khôi phục toàn Điều của các candidate được chọn. Critic sinh nháp, kiểm tra graph, lọc ứng viên và sinh lại có điều kiện.

Sự tách biệt offline/online có hai lợi ích. Thứ nhất, chi phí embedding và trích xuất graph được thực hiện trước, không làm tăng độ trễ của mọi câu hỏi. Thứ hai, nội dung văn bản và graph có thể được version độc lập với mô hình sinh. Tuy nhiên, nếu corpus thay đổi thì Qdrant và Neo4j phải được xây lại đồng bộ; nếu chỉ cập nhật một phía, Critic có thể tham chiếu ID không còn tồn tại.

3.4. Thiết kế dữ liệu văn bản



Hình 3.2: Vòng đời dữ liệu từ văn bản nguồn đến chỉ mục, đồ thị, testset và trả lời trực tuyến

Luồng dữ liệu bắt đầu từ văn bản DOCX, đi qua cùng một bộ phân tích cấu trúc để tránh chênh lệch giữa dữ liệu index và dữ liệu dùng kiểm định. Đầu ra của chunker tách thành ba nhánh có liên hệ bằng ID: child/parent phục vụ Qdrant; parent Article

phục vụ trích xuất Neo4j; và toàn văn Điều phục vụ đối chiếu dieu_ids của testset. Mỗi lần cập nhật corpus cần gắn version cho cả ba nhánh. Nếu chỉ tái tạo vector nhưng giữ graph hoặc testset theo snapshot cũ, các ID vẫn có thể hợp lệ về cú pháp nhưng sai về nội dung và hiệu lực.

3.4.1. Tiền xử lý

Chunker đọc các đoạn theo thứ tự xuất hiện trong DOCX, loại đoạn rỗng và dừng tại vùng ghi chú cuối văn bản đã nhận dạng. Tên file được chuẩn hóa thành van_ban_id; bộ regex nhận diện tiêu đề Điều <số>, Khoản dạng số và Điểm dạng chữ cái. Trạng thái phân cấp hiện hành được cập nhật khi gặp tiêu đề mới để mỗi đoạn nội dung mang đúng vị trí pháp lý. Một lời dẫn Điều chỉ chứa tiêu đề được ghép vào child nội dung đầu tiên khi có thể; nếu không, nó trở thành preamble child riêng. Với đoạn vượt giới hạn, thứ tự ưu tiên tách là đoạn văn, câu rồi từ; hậu tố split được thêm vào ID để không mất khả năng truy vết.

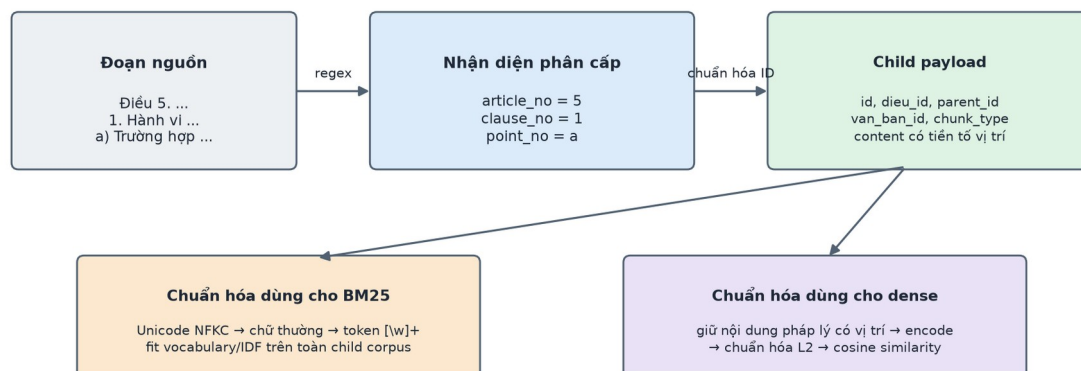
3.4.2. Cơ chế parent/child

Child chunk là đơn vị truy xuất vì tập trung vào một lời dẫn, Khoản hoặc Điểm, giảm nguy cơ một embedding phải đại diện cho nhiều chủ đề. Trước khi encode, nội dung child được gắn tiền tố vị trí gồm văn bản, Điều, Khoản và Điểm; cùng nội dung như “phạt tiền” nhờ đó có ngữ cảnh pháp lý phân biệt. Parent chunk là toàn văn một Điều theo đúng thứ tự nguồn và là đơn vị khôi phục ngữ cảnh. Mọi child có parent_id trỏ tới parent tương ứng. Với article expansion, hệ thống lấy parent_id từ child hit; với critic, graph đề xuất dieu_id cần bổ sung nhưng văn bản bằng chứng vẫn được đọc từ parent collection, không sinh trực tiếp từ nội dung graph.

3.4.3. Kiểm soát trùng và va chạm ID

Các văn bản có thể lặp số Điều trong phụ lục hoặc bản hợp nhất. Hậu tố _O<n> được dùng cho occurrence trùng số; _C<n> xử lý collision của child; _S<n> đánh dấu split. Cơ chế này tránh ghi đè trong Qdrant và giữ khả năng truy ngược về đoạn nguồn.

CHUẨN HÓA VĂN BẢN VÀ TẠO BIỂU DIỄN TRUY XUẤT



Hình 3.3: Chuẩn hóa cấu trúc và biểu diễn truy xuất dense/BM25

Chuẩn hóa không đồng nghĩa với xóa dấu hoặc rút gọn nội dung pháp luật. Phần văn bản cung cấp cho dense encoder được giữ nguyên ý nghĩa và thêm vị trí pháp lý; chỉ nhánh BM25 áp dụng Unicode NFKC, chuyển chữ thường và tách token bằng biểu thức $[\wedge]^+$. Vocabulary và IDF phải được fit một lần trên toàn child corpus rồi tái sử dụng ở thời điểm query. Dense vector được chuẩn hóa L2 trước khi tìm kiếm cosine. Hai nhánh do đó có phép chuẩn hóa khác nhau nhưng cùng tham chiếu một child_id và payload.

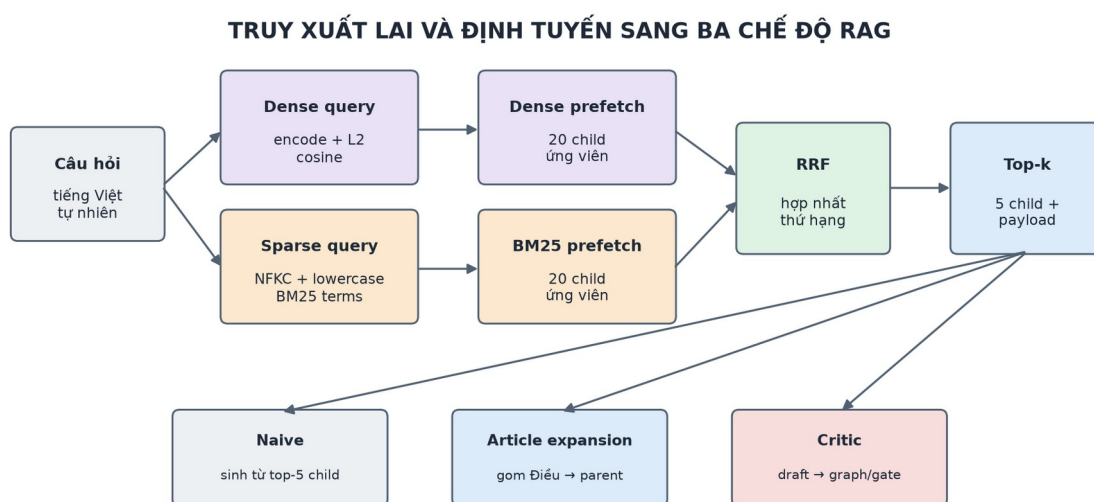
3.5. Thiết kế chỉ mục Qdrant

Collection legal_child_chunks lưu named vectors dense và bm25, cùng payload gồm ID, nội dung, cấu trúc và parent. Collection legal_parent_chunks lưu toàn văn từng Điều. Chỉ mục Base hiện có 10.355 child points, 1.211 parent points và dense cosine 1.024 chiều. BM25 index được fit trên toàn bộ child corpus với doc_count=10.355, vocab_size=1.914, avgdl $\approx$ 57,5987, $k_1=1,5$ và $b=0,75$, rồi lưu để query encoder dùng đúng vocabulary và IDF.

Tập bằng chứng Base có hash metadata 8a8248...91a5; tập BM25 có hash 59ed...18d1. Các chuỗi rút gọn này được dùng để nhận diện snapshot, không thay cho manifest bất biến đầy đủ. Collection GTE hiện hữu bị loại khỏi đánh giá chính vì kích

thước 1.024 không khớp 768 chiều của model card [15]; dùng collection đó để suy luận kết quả GTE sẽ không bảo đảm đúng mô hình.

Ở truy vấn, model embedding và BM25 vectorizer được load một lần khi khởi tạo pipeline. Chuỗi xử lý gồm: (1) nhận nguyên văn câu hỏi; (2) dense encoder tạo vector chuẩn hóa L2; (3) BM25 vectorizer áp dụng đúng pipeline NFKC/chữ thường/token và vocabulary đã fit; (4) Qdrant chạy hai prefetch độc lập, mặc định mỗi nhánh lấy 20 candidate; (5) RRF hợp nhất theo thứ hạng; (6) trả top-5 child cùng payload; và (7) ánh xạ parent_id/dieu_id cho mode downstream. Việc tái sử dụng model, client và BM25 giúp tránh thời gian load lại cho mỗi câu hỏi.



Hình 3.4: Pipeline truy xuất lai dense/BM25, RRF và ba chế độ RAG

Bảng 3.2. Các bước xử lý một truy vấn từ đầu vào đến ngữ cảnh sinh

Bước	Đầu vào	Xử lý	Đầu ra/kiểm soát
1	Chuỗi người dùng	Loại input rỗng; router phân loại hội thoại/pháp luật	chit_chat trả trực tiếp; legal đi tiếp
2	Câu hỏi pháp luật	Dense encode bằng Base và chuẩn hóa L2	Vector 1.024 chiều đúng collection

Bước	Đầu vào	Xử lý	Đầu ra/kiểm soát
3	Cùng câu hỏi	NFKC, lowercase, token hóa BM25	Sparse indices/values theo vocabulary corpus
4	Hai vector query	Prefetch dense và sparse trên child collection	Hai danh sách tối đa 20 candidate
5	Hai bảng hạng	Qdrant FusionQuery RRF	Top-5 child đã hợp nhất
6	Child payload	Lấy dieu_id, parent_id, score và content	Context cơ sở, danh sách Điều truy xuất
7	Mode	Giữ child, mở rộng parent hoặc critic graph/gate	Final context và answer có trace

Bảng 3.3. Payload tối thiểu của child và parent chunk

Trường	Child	Parent	Ý nghĩa
id	Có	Có	Định danh duy nhất
dieu_id	Có	Có	Khóa nhóm theo Điều
parent_id	Có	Không bắt buộc	Liên kết child-parent
van_ban_id	Có	Có	Định danh văn bản
chunk_type	Có	Parent	Loại cấu trúc
content	Có	Có	Nội dung truy xuất hoặc toàn Điều
dense	Có	Tùy cấu hình	Vector ngữ nghĩa
bm25	Có	Không	Vector thưa

3.6. Thiết kế Knowledge Graph

3.6.1. Trích xuất hai lượt

Parent Article được đưa vào Qwen2.5-7B-Instruct. Pass 1 trích xuất entity với giới hạn 2.048 token; Pass 2 nhận danh sách ID entity rồi trích xuất relation với giới hạn 1.024 token. Nhiệt độ bằng 0 và tối đa ba lần thử. Model được lượng tử hóa NF4 4-bit để phù hợp GPU T4 16 GB.

Tách entity và relation làm hai lượt giảm kích thước JSON trong mỗi lần sinh và giảm trường hợp output bị cắt giữa chừng. Hàm sửa JSON tìm phần đóng ngoặc hợp lệ gần nhất để giữ phần dữ liệu đã hoàn thành. Đây là biện pháp phục hồi cú pháp, không bảo đảm nội dung được trích xuất là đúng.

3.6.2. Chuẩn hóa và placeholder

Graph builder chuyển ID sang dạng không dấu, chữ thường và gán tiền tố văn bản. Nút và cạnh được deduplicate trước khi nạp. Với THAM_CHIEU, nếu đích chưa có, builder tạo placeholder is_placeholder=True. Placeholder cho phép graph duy trì thông tin “có một dẫn chiếu” nhưng cần được giải quyết trước khi dùng cho production.

3.6.3. Lược đồ graph

Bảng 3.4. Các nhóm nút và quan hệ chính trong đồ thị

Nhóm	Nhãn/quan hệ	Mục đích
Cấu trúc	Dieu, Khoan, Diem	Bảo toàn phân cấp pháp luật
Nội dung	HanhVi, CheTai, ChuThe, NghiaVu	Mô tả đối tượng và quy tắc
Cạnh cấu trúc	CO_KHOAN, CO_DIEM	Duyệt thành phần cùng Điều
Cạnh chế tài	CHE_TAI_CHINH, CHE_TAI_BO_SUNG	Phát hiện chế tài kép

Nhóm	Nhãn/quan hệ	Mục đích
Cạnh dẫn chiếu	THAM_CHIEU	Tìm Điều liên quan ngoài top-k
Cạnh quy định	QUY_DINH_HANH_VI	Liên kết cấu trúc với hành vi

3.7. Thiết kế ba chế độ RAG

3.7.1. Naive RAG

Naive là baseline. Pipeline lấy top-5 child chunks và gọi generator bằng prompt chung. Mode này không khôi phục parent và không truy vấn graph. Ưu điểm là context nhỏ và luồng đơn giản; nhược điểm là dễ thiếu Khoản hoặc Điều liên quan nằm ngoài top-5.

3.7.2. Article expansion

Mode mở rộng quét pool tối đa 20 child candidates, gom score theo Điều bằng giá trị child cao nhất, giữ các Điều đạt ít nhất 40% score Điều đứng đầu, lấy tối đa năm Điều và sắp toàn văn giảm dần theo score. Mode không dùng graph và không theo THAM_CHIEU. Các giá trị 0,60 trong source snapshot cũ bị thay bởi default 0,40 của mã nguồn hiện hành.

3.7.3. KG-based critic

Critic bắt đầu từ top-5 child và câu trả lời nháp. Các Điều có score ít nhất 60% score cao nhất được đưa vào kiểm tra, tối đa bốn Điều. Engine so sánh số phần đã lấy với tổng phần theo Qdrant/Neo4j, tìm chế tài kép và duyệt THAM_CHIEU ra/vào; traversal nhiều bước giới hạn max_hops=2 và tổng số Điều lấy qua graph tối đa tám. Đối với cross-Article, gate đánh giá parent ứng viên; với same-Article, gate đánh giá các anchor child và chấp nhận Điều nếu ít nhất một anchor liên quan. Candidate được chấp nhận đóng góp toàn parent Article.

Nếu có candidate, hệ thống nối chúng vào context gốc và sinh lại từ đầu. Nếu không, draft là final answer. Một safeguard riêng lập parent Article đứng đầu và sinh

lại khi initial top-k chứa từ hai văn bản trở lên nhưng graph không lấy được Điều mới. Exception của gate hiện được xử lý theo fail-open: candidate được chấp nhận để thiên về recall. Đây là hành vi đã xác minh từ code, đồng thời là rủi ro vì lỗi kỹ thuật chưa được tách khỏi quyết định “yes”.

Bảng 3.5. So sánh vận hành của ba chế độ

Tham số	Naive RAG	Article expansion	KG-based critic
Candidate retrieval	Top-5 child	Top-20 child	Top-5 child
Số Điều tối đa	Không áp dụng	5	4 để kiểm tra
Ngưỡng tương đối	Không áp dụng	0,40	0,60
Toàn văn Điều	Không	Có	Có, khi được chấp nhận
Truy vấn Neo4j	Không	Không	Có
Semantic gate	Không	Không	Yes/No
Sinh nháp	Không tách	Không tách	Có
Sinh lại có điều kiện	Không	Không	Có

3.8. Thuật toán Critic Agent

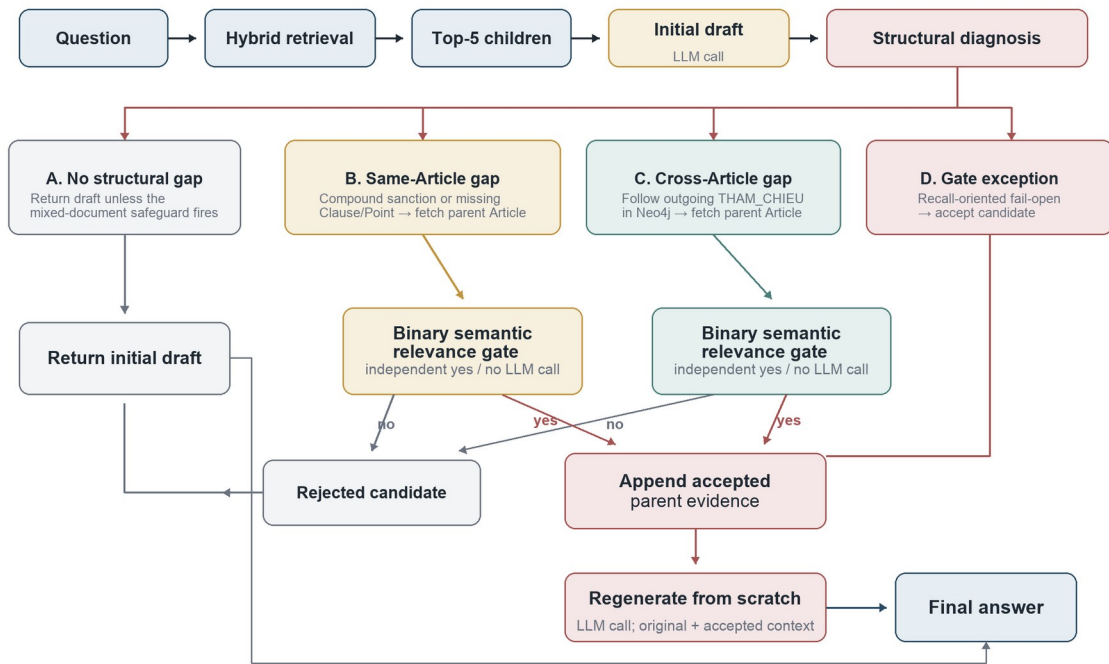
Thuật toán 1 mô tả đúng thứ tự xử lý của workflow hiện hành. Mã giả phân biệt Article đã xuất hiện dưới dạng child với parent Article đã được bổ sung; same-Article vẫn có thể được chọn để khôi phục toàn văn, còn ứng viên cross-Article đã có trong top-k bị loại khỏi tập thiếu.

Thuật toán 1. KG-based Critic Evidence Repair

```

INPUT: query q; top-k child chunks Ck; ranked Article scores S;
       Qdrant Q; Neo4j G
OUTPUT: final_answer; critic_trace
1  draft <- GENERATE(q, Ck)
2  Aall <- UNIQUE_ARTICLES(Ck)
3  Afocus <- first at most 4 Articles a with S[a] >= 0.60 * max(S)
4  if Afocus is empty then return (draft, empty_trace)
5  retrieved_parts <- COUNT_UNIQUE_CHILD_PARTS(Ck, by Article)
6  total_parts[a] <- max(Q.child_count(a), G.structure_count(a))
7  gaps <- G.COMPOUND_SANCTIONS(Afocus)
8      union STRUCTURAL_GAPS(Afocus, retrieved_parts, total_parts)
9      union G.THAM_CHIEU_NEIGHBORS(Afocus, both directions)
10 candidates <- ORDER_AND_DEDUPLICATE(gaps)
11 Remove cross-Article candidates already represented in Aall;
    keep same-Article candidates for parent reconstruction
12 accepted <- []; rejected <- []; fail_open <- []
13 for each candidate a in candidates do
14     parent <- Q.FETCH_PARENT(a)
15     gate_input <- retrieved child anchors if a is same-Article,
                    otherwise parent
16     try relevant <- SEMANTIC_GATE(q, gate_input)
17     catch gate_exception:
18         relevant <- true; APPEND(fail_open, a)
19     if not relevant then APPEND(rejected, a); continue
20     if parent exists then APPEND(accepted, (a, parent))
21 frontier <- Article IDs in accepted; known <- Aall union frontier
22 repeat at most 2 rounds while frontier is nonempty and |accepted| < 8:
23     next <- G.THAM_CHIEU_NEIGHBORS(frontier, both directions) - known
24     gate each parent in next by Lines 14-20; update known and frontier
25 if accepted is empty and Ck spans at least 2 source documents then
26     top_parent <- Q.FETCH_PARENT(first Article in Aall)
27     if top_parent exists then APPEND(accepted, safeguard(top_parent))
28 if accepted is empty then final_answer <- draft
29 else final_answer <- REGENERATE(q, Ck, accepted evidence)
30 critic_trace <- {gaps, accepted IDs, rejected IDs, fail_open,
                    safeguard, regenerated, token usage}
31 return (final_answer, critic_trace)

```



Hình 3.5: Lưu đồ quyết định của Critic: phát hiện thiếu, semantic gate và sinh lại có điều kiện

Bảng quyết định sau phân biệt hành vi đã cài đặt và nhánh chỉ được đề xuất. Không phát hiện gap dẫn tới trả draft, trừ trường hợp safeguard nhiều văn bản. Gap cùng Điều hoặc THAM_CHIEU tạo candidate parent; gate yes hoặc exception làm candidate được chấp nhận, gate no làm candidate bị loại và ID được ghi vào báo cáo. Khi toàn bộ candidate bị loại, hệ thống chỉ bổ sung Điều đứng đầu mà không gọi gate nếu top-k ban đầu trải trên ít nhất hai văn bản; nếu điều kiện safeguard này không thỏa, hệ thống giữ draft. Query rỗng bị CLI bỏ qua; mode không hợp lệ bị argparse từ chối. Mã hiện hành có try/except cục bộ quanh gate và một số truy vấn graph, nhưng chưa có circuit breaker hoặc thông báo người dùng chuẩn hóa cho mọi lỗi Qdrant, Neo4j và LLM; các cơ chế đó là hướng phát triển, không phải chức năng đã hoàn tất.

Bảng 3.6. Bảng quyết định các nhánh vận hành đã xác minh và đề xuất

Tình huống	Hành vi hiện hành	Trạng thái
Không có evidence gap	Trả draft; có thể kích hoạt safeguard nếu top-k chứa nhiều văn bản	Đã xác minh từ code
Thiếu Khoản/Điểm, chế tài bổ sung hoặc khắc phục trong cùng Điều	Lấy parent Article, chạy gate, sinh lại nếu chấp nhận	Đã xác minh từ code
Có THAM_CHIEU sang Điều khác	Duyệt graph tối đa hai hop, lấy parent, gate từng candidate	Đã xác minh từ code
Gate trả yes	Thêm parent vào context được chấp nhận	Đã xác minh từ code
Gate trả no	Loại candidate, ghi ID bị từ chối	Đã xác minh từ code
Gate exception	Fail-open, chấp nhận candidate	Đã xác minh; rủi ro
Không có candidate được chấp nhận	Nếu top-k thuộc từ hai văn bản trở lên, thêm parent đứng đầu bằng safeguard và sinh lại; nếu không, giữ draft	Đã xác minh từ code
Câu hỏi rỗng/mode sai ở CLI	Bỏ qua dòng rỗng/argparse từ chối mode	Đã xác minh từ code

Tình huống	Hành vi hiện hành	Trạng thái
Câu hỏi ngoài corpus	Có thể vẫn sinh từ context gần nhất; chưa có abstention chắc chắn	Giới hạn hiện tại
Mất Qdrant/Neo4j/LLM	Chưa có UX và policy nhất quán cho mọi điểm lỗi	Hướng phát triển

Thiết kế cố ý không cho graph tạo câu trả lời. Graph chỉ đề xuất vị trí thiếu và lý do; generator vẫn đọc văn bản đầy đủ. Điều này giúp lỗi relation của graph ít trực tiếp biến thành nội dung trả lời hơn, dù vẫn có thể làm hệ thống bỏ sót hoặc lấy thừa candidate.

3.9. Thiết kế bộ đánh giá

Bộ eval_testset.jsonl có 301 record, mỗi record gồm id, category, dieu_ids, van_ban, question, required_facts và reference_answer. Bốn nhóm được thiết kế để tách các loại khó khăn: 51 câu chế tài kép trong cùng Điều, 16 câu tham chiếu chéo, 130 câu đa thành phần cấu trúc và 104 câu đối chứng không có khoảng trống.

Bảng 3.7. Phân bố bộ câu hỏi đánh giá

Nhóm	Số câu	Khó khăn cần đo
same_dieu_compound_penalty	51	Phạt chính, bổ sung và khắc phục ở các phần khác nhau
cross_reference	16	Cần theo dẫn chiếu sang Điều khác
structural_multi_part	130	Cần nhiều Khoản/Điểm cùng Điều
control_no_gap	104	Ngữ cảnh top-k đã tương đối đầy đủ
Tổng	301	

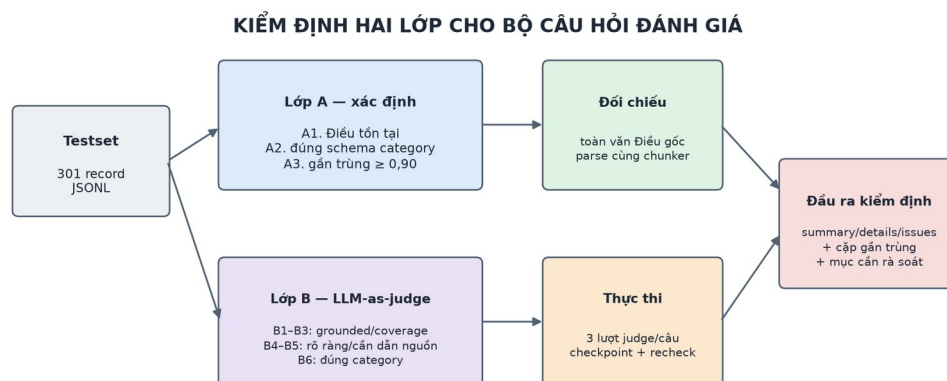
Base retriever chạy cả ba chế độ, tạo ba cấu hình đầu-cuối. Kết quả theo từng câu cần chứa câu trả lời, ngữ cảnh truy xuất, xếp hạng Điều, Điều lấy thêm từ đồ thị, bản

nhập và báo cáo Critic. Tập kết quả ngày 31/07/2026 có 301 bản ghi cho Naive và 301 bản ghi cho Article Expansion; artefact Critic hiện lưu là một tập con có đúng schema evaluation. Repository không có log đủ để kết luận toàn bộ run Critic chỉ xử lý số câu tương ứng với tập con này. Vì chưa có raw Critic đủ 301 câu và raw judge/checkpoint đầy đủ, phần kết quả tách rõ chỉ số có thể tái tính từ artefact hiện có với chỉ số tổng hợp do tài liệu nguồn báo cáo; kiểm định cặp không được thực hiện do thiếu đầy đủ các cặp kết quả và repeated runs.

3.9.1. Kiểm định xác định và đối chiếu toàn văn Điều

Nguồn tài liệu phương pháp kiểm định testset mô tả một quy trình kiểm định độc lập với bước chạy RAG. Để tránh “tự kiểm tra bằng chính dữ liệu đáp án”, mỗi `diều_id` được ánh xạ về toàn văn Điều gốc được parse bằng cùng VBPLChunker; judge nhận phần văn bản này cùng câu hỏi, required facts và reference answer. Lớp A không dùng mô hình ngôn ngữ: A1 kiểm tra mọi `diều_id` tồn tại; A2 kiểm tra nhóm `cross_reference` có ít nhất hai ID và ba nhóm còn lại có đúng một ID; A3 tìm câu hỏi gần trùng bằng `difflib.SequenceMatcher` với ngưỡng 0,90.

Lớp B dùng LLM-as-a-judge. Ba tiêu chí về tám nguồn được hỏi chung trong lượt thứ nhất, hai tiêu chí ngôn ngữ/citation trong lượt thứ hai và tiêu chí phân nhóm trong lượt thứ ba. Tách ba lượt giúp prompt ngắn hơn và cho phép chạy lại riêng B6 bằng `--recheck-category`; checkpoint được ghi sau mỗi câu để tiếp tục khi gián đoạn. Cách đánh giá tự động này phù hợp với hướng RAGAS, ARES và LLM-as-a-judge [12]–[14], nhưng vẫn cần đối chiếu người thật trước khi xem là xác nhận pháp lý.



Hình 3.6: Quy trình kiểm định hai lớp cho testset và các tệp bằng chứng đầu ra cần lưu

Bảng 3.8. Chín kiểm tra dùng để kiểm định bộ testset

Mã	Loại	Câu hỏi kiểm định	Điều kiện đạt
A1	Xác định	Mọi <code>diem_id</code> có trong corpus?	Không có ID không ánh xạ được
A2	Xác định	Số Điều có phù hợp category?	Cross-reference ≥ 2 ; nhóm khác =1
A3	Xác định	Có câu hỏi gần trùng?	Gần cở cặp similarity $\geq 0,90$ để rà soát
B1	Judge	Required facts có bám toàn văn Điều?	Mỗi fact được văn bản hỗ trợ
B2	Judge	Reference answer có bám toàn văn Điều?	Không thêm kết luận không có căn cứ
B3	Judge	Đáp án có bao phủ required facts?	Không bỏ ý bắt buộc

Mã	Loại	Câu hỏi kiểm định	Điều kiện đạt
B4	Judge	Câu hỏi có tự nhiên, rõ ràng?	Có thể hiểu mà không cần ngữ cảnh ẩn
B5	Judge	Câu hỏi có thật sự cần dẫn nguồn?	Không thể trả lời đáng tin chỉ bằng kiến thức chung
B6	Judge	Nhân category có đúng cơ chế cần đo?	Phù hợp định nghĩa của một trong bốn nhóm

3.9.2. Chính sách diễn giải kết quả kiểm định

Kết quả lớp A có thể tái lập từ scripts/validate_testset.py, corpus và testset đúng phiên bản dữ liệu; A1/A2 cho 0 lỗi và A3 gần cờ hai cặp gần trùng. Kết quả lớp B còn phụ thuộc mô hình, phiên bản, prompt và chính sách thử lại. Dự án không có checkpoint hay các tệp tổng hợp, chi tiết và lỗi của lần chạy B1-B6, nên Chương 5 báo cáo tỷ lệ và ID đúng theo tài liệu nguồn, không tuyên bố đã chạy lại. Hai cặp gần trùng không tự động bị xóa: chúng chỉ bị loại nếu rà soát cho thấy cùng ý định và cùng bằng chứng.

3.10. Thiết kế bảo mật và quản trị dữ liệu

Neo4j password, API key và endpoint được đọc từ biến môi trường; giá trị REPLACE_ME chỉ là placeholder. Service account cần nguyên tắc quyền tối thiểu. Dữ liệu pháp luật là công khai nhưng lịch sử truy vấn người dùng có thể phản ánh vấn đề pháp lý nhạy cảm; log production phải tách định danh, giới hạn retention và không dùng để huấn luyện nếu chưa có cơ sở pháp lý.

Quản trị corpus cần manifest chứa URL nguồn chính thức, ngày tải, ngày hiệu lực, trạng thái hết hiệu lực, mã băm tệp và phiên bản chunker. Quá trình đối chiếu dữ liệu phát hiện nhiều văn bản cần thay thế hoặc cập nhật sau ngày 01/07/2026. Vì vậy, hệ thống phải hiển thị ngày chụp corpus và không diễn giải câu trả lời như quy định hiện hành nếu chưa kiểm tra phiên bản.

4. CÀI ĐẶT VÀ THỰC NGHIỆM

4.1. Tổ chức mã nguồn

Mã nguồn hiện hành được chia theo trách nhiệm: src/data_processing cho chunking; src/data_ingestion cho Qdrant; src/retrieval cho BM25 và hybrid search; src/knowledge_graph cho extractor, builder và Neo4j; src/agents cho các agent thành phần; src/workflow nối các node bằng state chung; scripts cho ingest, chatbot, đánh giá và kiểm định testset. Hai entry point sản phẩm là scripts/run_chatbot.py (CLI) và app.py (Chainlit).

Bảng 4.1. Các module chính của hệ thống

Module	Tập tiêu biểu	Trách nhiệm
Data processing	src/data_processing/chunking.py	Điều - Khoản - Điểm, parent/child
Retrieval	bm25_sparse.py, qdrant_hybrid_search.py	Sparse/dense và RRF
KG extraction	entity_extractor.py, graph_builder.py	Entity, relation, chuẩn hóa
KG storage	neo4j_ingest.py	Nạp và truy vấn Neo4j
Agent/workflow	src/workflow/pipeline.py, src/agents/agent_critic/critic_query.py	Router, ba mode, completeness check và state
Product	scripts/run_chatbot.py, app.py	CLI, compare-all và giao diện Chainlit
Evaluation	run_evaluation.py, score_evaluation.py, validate_testset.py	Thu output, LCR/RAGAS, retrieval metrics và kiểm định benchmark

4.2. Môi trường và mô hình

Pipeline dùng Python, Qdrant embedded hoặc server, Neo4j và LLM qua endpoint tương thích OpenAI. CLI mặc định gọi qwen2.5:7b qua Ollama ở temperature 0,2. Chainlit mặc định chọn gpt-4o-mini qua proxy cấu hình trong code và còn cho phép Qwen2.5 7B, GPT-5-nano hoặc GPT-4.1-nano; việc tồn tại lựa chọn không chứng minh mọi model đã được dùng trong thí nghiệm. Graph extraction được source cũ mô tả dùng Qwen2.5-7B-Instruct; archive extraction hiện có nhưng môi trường và log trích xuất chưa đủ để chạy lại hoàn toàn.

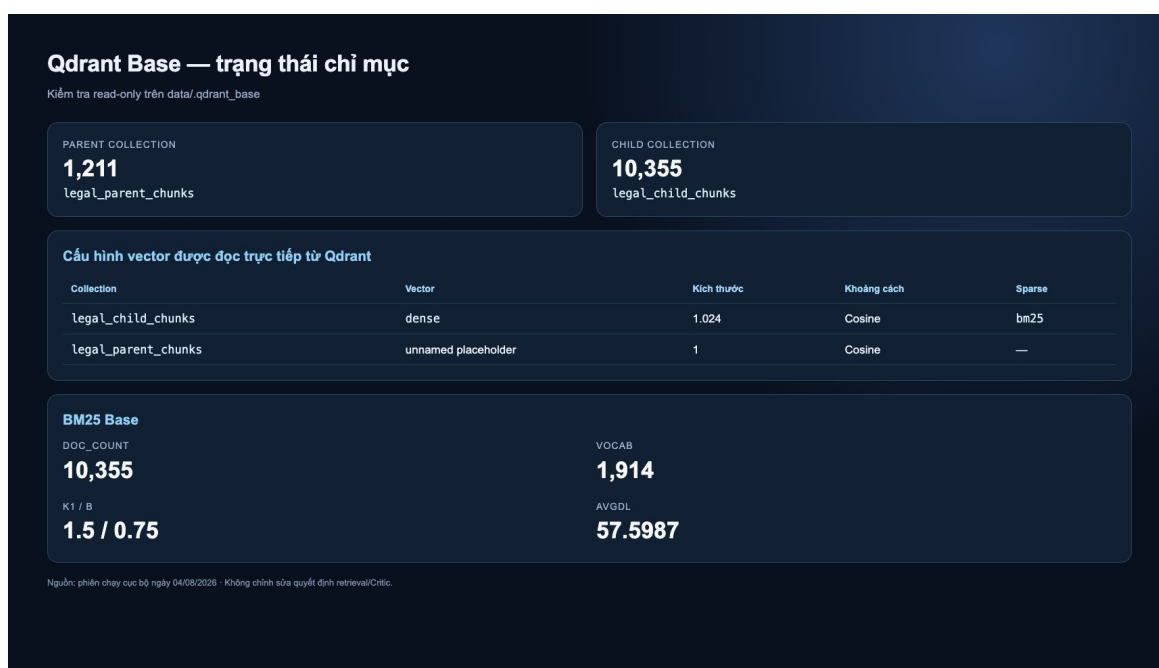
Bảng 4.2. Cấu hình thực nghiệm chính

Thành phần	Giá trị
Dense model chính	AITeamVN/Vietnamese_Embedding_v2
Dense dimension	1.024, cosine, chuẩn hóa L2
Sparse model	BM25, $k_1=1,5$, $b=0,75$
Fusion	RRF
Top-k child	5
Candidate pool của expand	20
Ngưỡng expand	0,40 của score cao nhất
Số Điều expand tối đa	5
Ngưỡng critic	0,60 của score cao nhất
Số Điều critic kiểm tra tối đa	4
Duyệt THAM_CHIEU	Tối đa 2 hop, tối đa 8 Điều fetch
Generator/gate CLI	qwen2.5:7b, temperature 0,2
Judge/RAGAS evaluator	Được tài liệu kết quả/phương pháp báo cáo; chưa tái chạy đầy đủ

4.3. Corpus và chỉ mục

Repository hiện chứa 23 tài liệu trong data/keep: 18 DOCX và năm PDF. Chạy lại VBPLChunker trên chính 18 DOCX tạo 1.211 parent Articles và 10.355 child chunks, gồm 5.159 Khoản, 4.964 Điểm và 232 điều_preamble; có 1.211 parent_id duy nhất. Kết quả này đồng thời khớp số điểm trong hai collection Base. Con số 1.209 của manifest cũ bị loại khỏi mô tả chính vì không khớp phép đếm tái lập. Năm PDF chưa được parser xử lý nên không phải bằng chứng retrievable của các kết quả.

Quá trình ingest tạo điểm parent, huấn luyện BM25 trên toàn bộ văn bản child, sau đó mã hóa và upsert child theo lô. Tùy chọn --resume bỏ qua ID đã có; --recreate xóa và tạo lại collection, vì vậy phải chỉ rõ đường dẫn collection để tránh mất chỉ mục khác. Mã băm metadata của Base và BM25 đã được ghi trong quá trình đối chiếu. Tài liệu hướng dẫn tái lập cấu hình GTE vẫn hữu ích cho thao tác ingest đúng 768 chiều, nhưng collection GTE hiện tại không đáp ứng điều kiện đó và không được dùng trong kết quả chính. Kết quả kiểm tra chỉ mục được trình bày tại Hình 4.1.

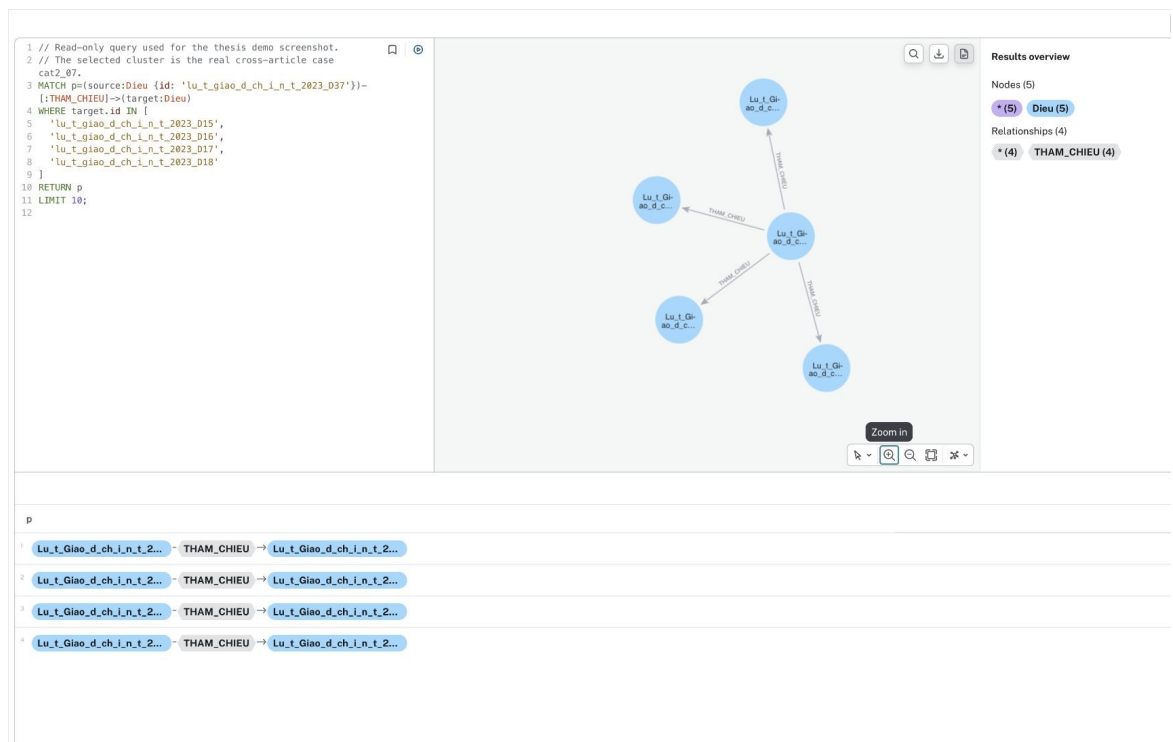


Hình 4.1: Kết quả kiểm tra chỉ mục Qdrant Base của corpus thực nghiệm.

4.4. Xây dựng và nạp đồ thị

Graph pipeline duyệt parent Article theo batch nhỏ để tránh hết bộ nhớ. Mỗi output extraction được lưu JSONL, cho phép debug và retry. Graph builder chuẩn hóa ID, loại node/cạnh trùng, sửa liên kết cross-document và tạo Cypher. Neo4j ingestor thực thi các batch query và tạo index/constraint khi cần.

Kho dữ liệu trích xuất cũ chứa 18 tệp JSONL với 1.200 bản ghi, nhưng quá trình đối chiếu phát hiện một tệp Giao dịch điện tử trùng byte với tệp Viễn thông và chứa ID viễn thông. Trong lần kiểm tra ngày 04/08/2026, Neo4j ở trạng thái healthy; truy vấn trực tiếp xác nhận 15.666 node, 14.137 quan hệ, 1.140 node Điều và 1.612 quan hệ THAM_CHIEU. Truy vấn Cypher chỉ đọc cho Điều 37 trả về 5 node Điều và 4 quan hệ THAM_CHIEU trong 322 ms. Sai lệch gần nhãn trong kho cũ vẫn là giới hạn chất lượng dữ liệu; các số đếm cấu trúc không đồng nghĩa với precision hoặc recall của quan hệ. Cụm quan hệ được minh họa tại Hình 4.2.



Hình 4.2: Minh họa quan hệ THAM_CHIEU giữa các Điều trong Neo4j.

4.5. Quy trình thao tác và các kịch bản đầu vào

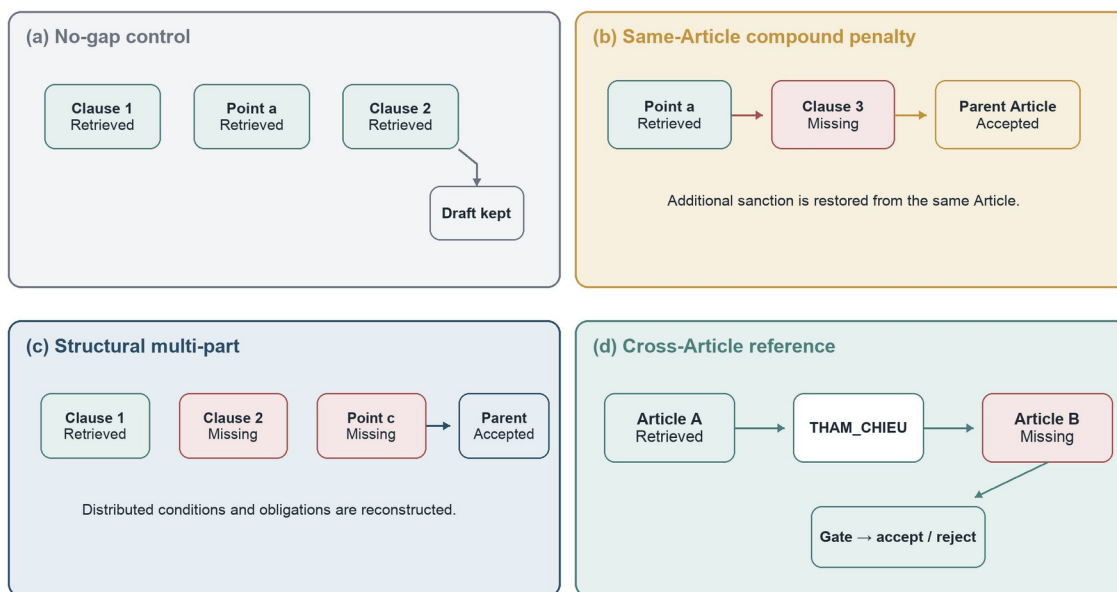
Sản phẩm hiện có hai điểm vào đã xác minh. `scripts/run_chatbot.py` là CLI phục vụ nghiên cứu: nhận `--query`, `--mode`, `--top-k`, `--compare-all` hoặc mở vòng lặp tương tác. `app.py` là giao diện Chainlit thực, có ba Chat Profile, đăng nhập và lịch sử cục bộ, bộ chọn embedding/LLM và bảng Critic Report bên cạnh câu trả lời. Tuy nhiên, giao diện chưa hiển thị đồng thời toàn bộ top-k payload, quyết định cố liên quan từ trace, danh sách Điều lấy thêm và token của từng lượt. Vì vậy, đây là nguyên mẫu CLI/Chainlit, không phải sản phẩm tư vấn pháp lý hoàn chỉnh. Giao diện chạy thực tế được trình bày tại Hình 4.4.

Quy trình khởi động theo mã nguồn hiện hành gồm: tạo `.env` từ mẫu và điền khóa nếu dùng mô hình qua API; chạy `docker compose up -d neo4j`; nạp đồ thị một lần bằng `docker compose --profile ingest up kg-ingest`; khởi động Chainlit bằng `docker compose up -d app`; sau đó mở địa chỉ cục bộ tại cổng 8000. Ollama chỉ cần khi chọn Qwen cục bộ. Trong lần kiểm tra ngày 04/08/2026, chuỗi thao tác này đã chạy đầu-cuối với `qwen2.5:7b`; Chainlit trả lời thành công và sau đó được dùng có chủ đích để giải phóng khóa Qdrant embedded.

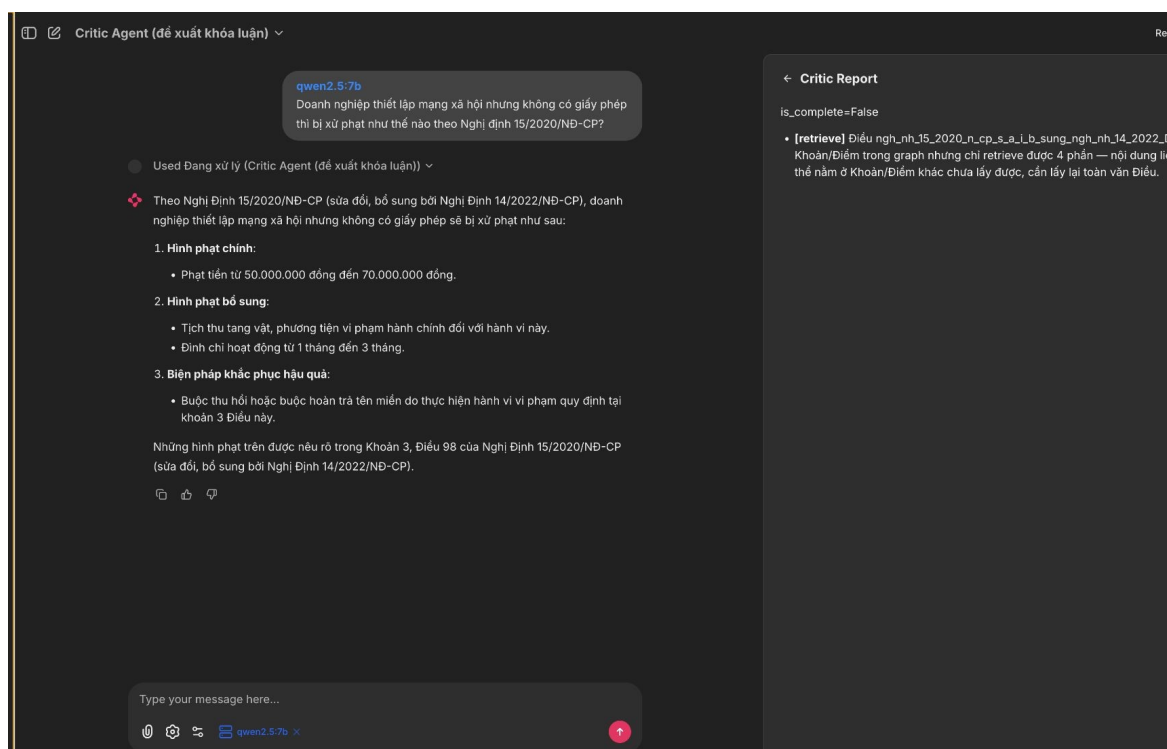
CLI Base được gọi rõ ràng bằng hai biến môi trường để cố định Base retriever thay cho cấu hình mặc định cũ trong phần mô tả của script:

```
QDRANT_PATH=data/.qdrant_base EMBEDDING_MODEL=AITeamVN/Vietnamese_Embedding_v2
python scripts/run_chatbot.py --query "..." --mode critic
```

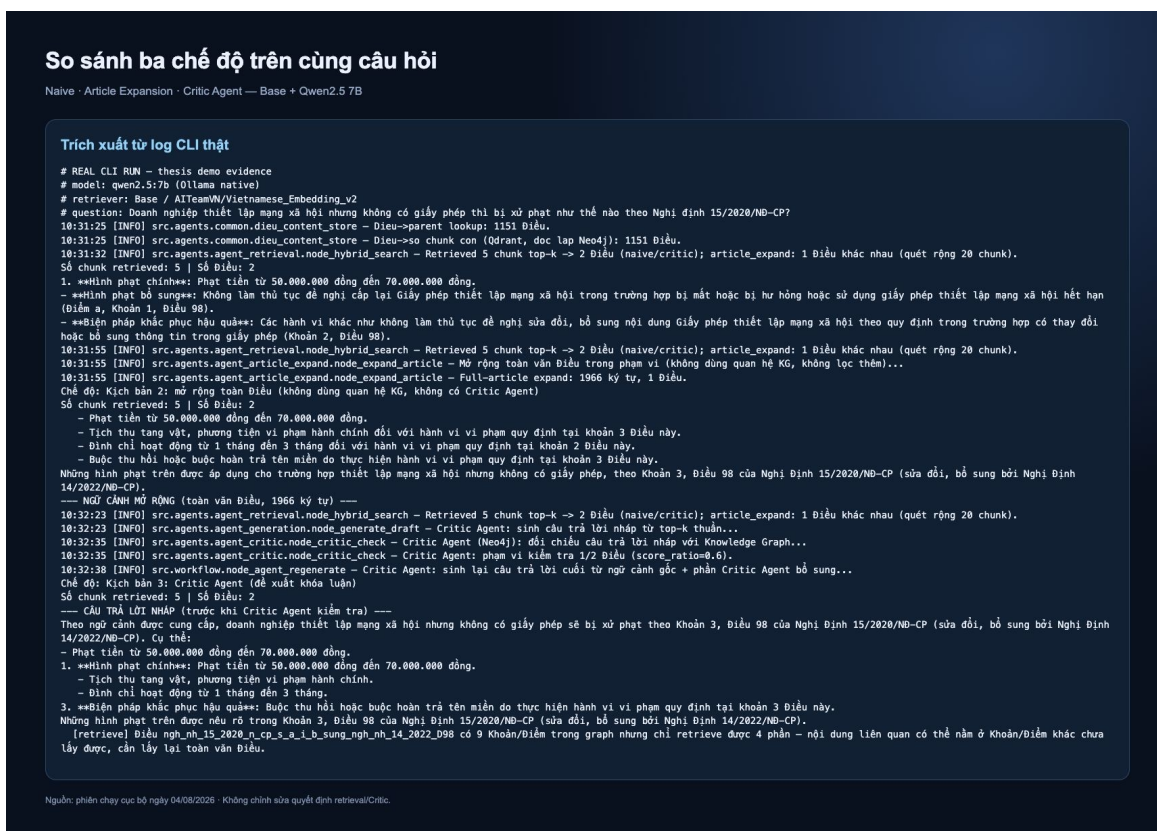
Để so sánh cùng một câu hỏi, dùng `--compare-all` với cùng `QDRANT_PATH` và `EMBEDDING_MODEL`. Script in câu hỏi, chế độ, số chunk, số Điều, bản nháp, báo cáo Critic và câu trả lời cuối. Các trường `graph_fetched_dieu_ids` và token có trong trạng thái nội bộ hoặc tệp đánh giá nhưng CLI chưa in đầy đủ. Kết quả so sánh ba chế độ và trace quyết định của Critic được trình bày tại Hình 4.5 và Hình 4.6.



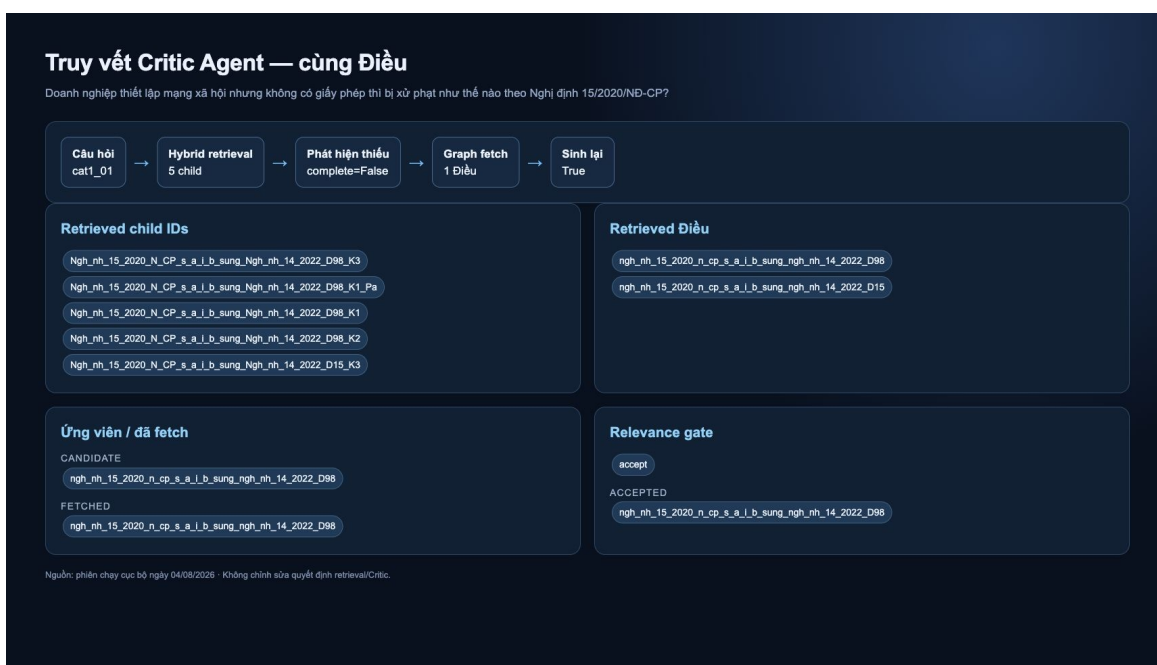
Hình 4.3: Bốn kiểu khoảng trống bằng chứng và hành vi của Critic trong nguyên mẫu



Hình 4.4: Giao diện nguyên mẫu Chainlit khi chạy chế độ KG-based Critic.



Hình 4.5: Kết quả so sánh ba chế độ trên cùng một câu hỏi bằng CLI.



Hình 4.6: Trace quyết định của Critic Agent từ kết quả chạy thực tế.

Bảng 4.3. Các kịch bản nhập câu hỏi và hành vi mong đợi

Kịch bản	Ví dụ/mệnh lệnh	Mode phù hợp	Hành vi mong đợi
Hội thoại thông thường	“Xin chào”	Bất kỳ trong phiên chat	Router trả lời xã giao, is_chit_chat=True; evaluation bỏ router
Một quy định đơn lẻ	“Mức phạt đối với hành vi X là bao nhiêu?”	naive	Trả lời từ top-k child; nhanh, dùng ít context
Nhiều phần trong cùng Điều	“Ngoài phạt tiền còn đình chỉ hay khắc phục gì?”	article_expand	Gom candidate theo Điều và đưa toàn parent Article vào context
Dẫn chiếu/điều kiện phức hợp	“Trường hợp nào áp dụng ngoại lệ được dẫn chiếu tại Điều khác?”	critic	Sinh draft, kiểm tra graph, gate parent liên quan rồi mới sinh lại
So sánh thuật toán	--query "..." --compare-all	Cả ba	Chạy cùng input qua ba mode và in kết quả tách biệt
Nhập nhiều câu liên tiếp	Không truyền --query	Tương tác	Hiện prompt, bỏ qua dòng rỗng, tiếp tục đến khi người dùng thoát
Điều chỉnh độ rộng retrieval	--top-k 8	Mode đã chọn	Lấy số child theo tham số; log phải ghi top-k để tái lập
Input/mode không hợp lệ	Query rỗng hoặc mode ngoài enum	Không áp dụng	Dòng rỗng bị bỏ qua; argparse từ chối mode ngoài lựa chọn

Kịch bản	Ví dụ/mệnh lệnh	Mode phù hợp	Hành vi mong đợi
Không có bằng chứng đủ	Câu hỏi ngoài corpus	Mode bất kỳ	Code chưa có abstention chắc chắn; cần coi câu trả lời là không đủ bằng chứng

Để đặt câu hỏi có chất lượng, người dùng nên nêu rõ chủ thể, hành vi, bối cảnh và thông tin cần biết. Chẳng hạn, “Doanh nghiệp bị xử lý thế nào khi thu thập dữ liệu cá nhân không có sự đồng ý, có bị đình chỉ không?” rõ hơn “Vi phạm dữ liệu bị sao?”. Dù vậy, code hiện chưa có nhánh hỏi lại có cấu trúc khi thiếu tình tiết; đây là yêu cầu tương lai.

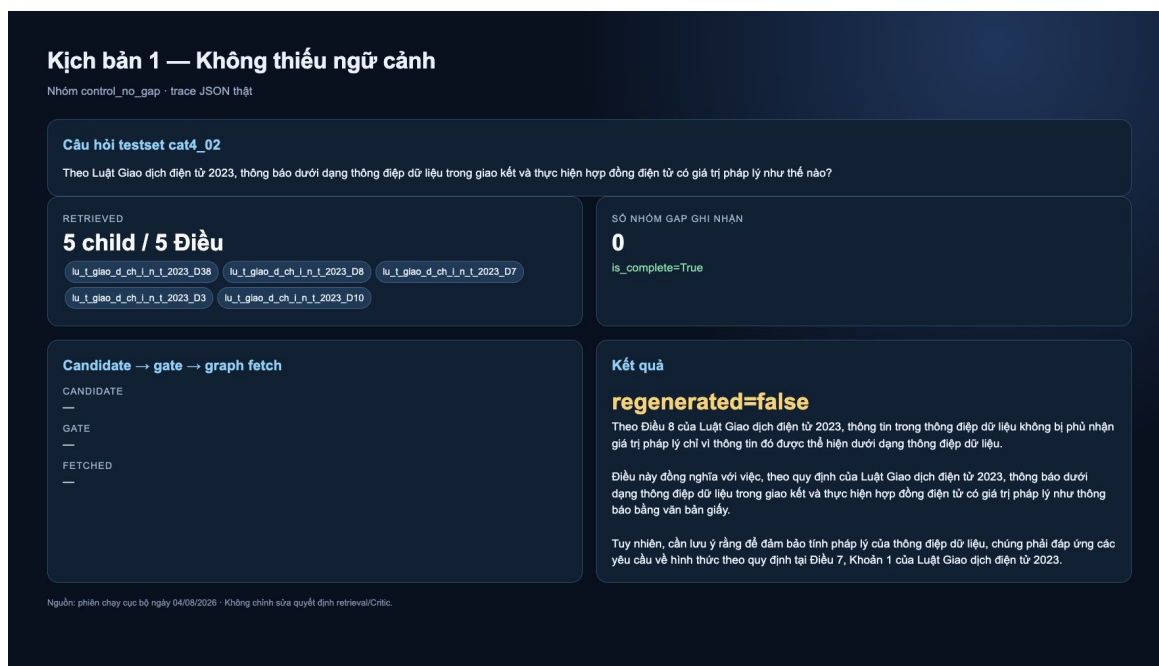
Sáu tình huống dưới đây dùng bằng chứng ở ba mức khác nhau và được ghi rõ để tránh suy diễn. Ba case đầu (`cat4\_02`, `cat1\_01`, `cat2\_07`) được chạy ngày 04/08/2026 bằng workflow production trên mã nguồn 9bf145e6744599e1e15c21e2011a02f7ecff87f5, Base retriever, Qwen cục bộ, top-k = 5 và skip_router = true; trace đầy đủ nằm trong repository. Hai nhánh gate-reject và safeguard dùng trực tiếp các dòng `cat1\_03` và `cat1\_02` của output Critic ngày 31/07/2026. Nhánh fail-open được kiểm tra riêng bằng fault injection xác định ngày 05/08/2026 trên câu hỏi thật `cat1\_02` và parent Article thật từ Qdrant; đây không phải một lỗi tự nhiên quan sát trong run production. Các case chỉ minh họa đường đi của thuật toán, không thay thế phép đánh giá đủ 301 câu.

Bảng 4.4. Case study không phát hiện khoảng trống bằng chứng (cat4_02)

Trường	Nội dung truy vết
Testset ID	cat4_02
Câu hỏi	Theo Luật Giao dịch điện tử 2023, thông báo dưới dạng thông điệp dữ liệu trong giao kết và thực hiện hợp đồng điện tử có giá trị pháp lý như thế nào?

Trường	Nội dung truy vết
Nhóm	control_no_gap
Cấu hình	KG-based Critic; Base retriever; top-k = 5; skip_router = true
Required facts	(1) Thông báo dưới dạng thông điệp dữ liệu trong giao kết và thực hiện hợp đồng điện tử có giá trị pháp lý như thông báo bằng văn bản giấy
Top-k child IDs	D38_preamble, D8_preamble, D7_K1, D3_K16, D10_preamble
Retrieved Article IDs	D38, D8, D7, D3, D10
Evidence gap	Không phát hiện khoảng trống; critic_report.is_complete = true.
Candidate Article IDs	Không có
Graph-fetched Article IDs	Không có
Gate decision	Không gọi cổng liên quan; không có ứng viên, không fail-open.

Trường	Nội dung truy vết
Draft answer rút gọn	Theo Điều 8 của Luật Giao dịch điện tử 2023, thông tin trong thông điệp dữ liệu không bị phủ nhận giá trị pháp lý chỉ vì thông tin đó được thể hiện dưới dạng thông điệp dữ liệu. Điều này đồng nghĩa với việc, theo quy định của Luật Giao dịch điện tử 2023, thông báo dưới dạng thông điệp dữ liệu trong giao kết và thực hiện hợp đồng điện tử có giá trị pháp lý như thông báo bằng văn bản giấy. Tuy nhiên, cần lưu ý rằng để đảm bảo tính pháp lý của thông điệp dữ liệu, chúng phải đáp ứng các yêu cầu về hình thức theo quy định tại Điều 7, Khoản 1 của Luật Giao dịch điện tử 2023.
Final answer rút gọn	Theo Điều 8 của Luật Giao dịch điện tử 2023, thông tin trong thông điệp dữ liệu không bị phủ nhận giá trị pháp lý chỉ vì thông tin đó được thể hiện dưới dạng thông điệp dữ liệu. Điều này đồng nghĩa với việc, theo quy định của Luật Giao dịch điện tử 2023, thông báo dưới dạng thông điệp dữ liệu trong giao kết và thực hiện hợp đồng điện tử có giá trị pháp lý như thông báo bằng văn bản giấy. Tuy nhiên, cần lưu ý rằng để đảm bảo tính pháp lý của thông điệp dữ liệu, chúng phải đáp ứng các yêu cầu về hình thức theo quy định tại Điều 7, Khoản 1 của Luật Giao dịch điện tử 2023.
Required facts được bao phủ	Đối chiếu thủ công trace: 1/1 fact được nêu; pipeline không ghi nhận bao phủ theo fact.
Nhận xét	Giữ nguyên câu trả lời nháp; không sinh lại; 1.048 token, 1 lượt gọi mô hình; không lỗi.



Hình 4.7: Case study không phát hiện khoảng trống bằng chứng.

Bảng 4.5. Case study bổ sung bằng chứng trong cùng Điều (cat1_01)

Trường	Nội dung truy vết
Testset ID	cat1_01
Câu hỏi	Doanh nghiệp thiết lập mạng xã hội nhưng không có giấy phép thì bị xử phạt như thế nào theo Nghị định 15/2020/NĐ-CP?
Nhóm	same_dieu_compound_penalty
Cấu hình	KG-based Critic; Base retriever; top-k = 5; skip_router = true
Required facts	(1) Phạt tiền từ 50.000.000 đồng đến 70.000.000 đồng; (2) Tịch thu tang vật, phương tiện vi phạm hành chính; (3) Buộc thu hồi hoặc buộc hoàn trả tên miền
Top-k child IDs	D98_K3, D98_K1_Pa, D98_K1, D98_K2, D15_K3

Trường	Nội dung truy vết
Retrieved Article IDs	D98, D15
Evidence gap	thiếu cấu trúc trong cùng Điều
Candidate Article IDs	D98
Graph-fetched Article IDs	D98
Gate decision	accept D98; không có reject và không fail-open.
Draft answer rút gọn	Theo ngữ cảnh được cung cấp, doanh nghiệp thiết lập mạng xã hội nhưng không có giấy phép sẽ bị xử phạt theo Khoản 3, Điều 98 của Nghị Định 15/2020/NĐ-CP (sửa đổi, bổ sung bởi Nghị Định 14/2022/NĐ-CP). Cụ thể: - Phạt tiền từ 50.000.000 đồng đến 70.000.000 đồng. Không có thông tin về hình phạt bổ sung hoặc biện pháp khắc phục hậu quả trong ngữ cảnh được cung cấp, nên chúng ta chỉ có thể xác định rằng doanh nghiệp sẽ bị áp dụng hình thức xử phạt tiền theo mức quy định trên.

Trường	Nội dung truy vết
Final answer rút gọn	Theo Nghị Định 15/2020/NĐ-CP (sửa đổi, bổ sung bởi Nghị Định 14/2022/NĐ-CP), doanh nghiệp thiết lập mạng xã hội nhưng không có giấy phép sẽ bị xử phạt như sau: 1. Hình thức chính: - Phạt tiền từ 50.000.000 đồng đến 70.000.000 đồng. 2. Hình thức bổ sung (nếu cần): - Tịch thu tang vật, phương tiện vi phạm hành chính. - Đình chỉ hoạt động từ 1 tháng đến 3 tháng. 3. Biện pháp khắc phục hậu quả: - Buộc thu hồi hoặc buộc hoàn trả tên miền nếu do thực hiện hành vi vi phạm quy định tại khoản 3 Điều này. Những hình thức xử phạt trên được nêu rõ trong Khoản 3, Điều 98 của Nghị Định 15/2020/NĐ-CP (sửa đổi, bổ sung bởi Nghị Định 14/2022/NĐ-CP).
Required facts được bao phủ	Đối chiếu thủ công: nháp 1/3 fact; câu cuối 3/3 fact. Pipeline không ghi nhận bao phủ theo fact.
Nhận xét	Điều 98 có 9 phần nhưng top-k chỉ lấy 4; lấy lại toàn Điều và sinh lại; 3.696 token, 3 lượt gọi; không lỗi.

Kịch bản 2 — Thiếu trong cùng Điều
Nhóm same_dieu_compound_penalty - trace JSON thật

Câu hỏi testset cat1_01
Doanh nghiệp thiết lập mạng xã hội nhưng không có giấy phép thì bị xử phạt như thế nào theo Nghị định 15/2020/NĐ-CP?

RETRIEVED
5 child / 2 Điều
ngh_nh_15_2020_n_cp_s_a_l_b_sung_ngh_nh_14_2022_D98
ngh_nh_15_2020_n_cp_s_a_l_b_sung_ngh_nh_14_2022_D15

SỐ NHÓM GÁP GHI NHẬN
1
is_complete=False

Candidate → gate → graph fetch
CANDIDATE
ngh_nh_15_2020_n_cp_s_a_l_b_sung_ngh_nh_14_2022_D98
GATE
accept
FETCHED
ngh_nh_15_2020_n_cp_s_a_l_b_sung_ngh_nh_14_2022_D98

Kết quả
regenerated=true
Theo Nghị Định 15/2020/NĐ-CP (sửa đổi, bổ sung bởi Nghị Định 14/2022/NĐ-CP), doanh nghiệp thiết lập mạng xã hội nhưng không có giấy phép sẽ bị xử phạt như sau:
1. **Hình thức chính:**
- Phạt tiền từ 50.000.000 đồng đến 70.000.000 đồng.
2. **Hình thức bổ sung (nếu cần):**
- Tịch thu tang vật, phương tiện vi phạm hành chính.
- Đình chỉ hoạt động từ 1 tháng đến 3 tháng.

Nguồn: phiên chạy cục bộ ngày 04/08/2028 - Không chỉnh sửa quyết định retrieval/Critic.

Hình 4.8: Case study bổ sung bằng chứng trong cùng Điều.

Bảng 4.6. Case study bổ sung Điều qua quan hệ THAM_CHIEU (cat2_07)

Trường	Nội dung truy vết
Testset ID	cat2_07
Câu hỏi	Khi tôi gửi đơn đặt hàng điện tử cho một cửa hàng trực tuyến nhưng không thấy họ phản hồi xác nhận, tôi có thể coi như đơn hàng đó chưa được xác lập không? Và nói chung khi nào một thông điệp dữ liệu như đơn đặt hàng được xem là bên kia đã 'nhận' được về mặt pháp lý?
Nhóm	cross_reference
Cấu hình	KG-based Critic; Base retriever; top-k = 5; skip_router = true

Trường	Nội dung truy vết
Required facts	(1) Việc nhận thông điệp dữ liệu trong giao kết, thực hiện hợp đồng điện tử được thực hiện theo quy định tại Điều 16 của Luật (Điều 37); (2) Nếu không có thỏa thuận khác, người nhận được xem là đã nhận được thông điệp dữ liệu nếu thông điệp đó được nhập vào hệ thống thông tin do người nhận chỉ định và có thể truy cập được (Điều 16 khoản 2 điểm a); (3) Nếu trước hoặc trong khi gửi, người khởi tạo đã tuyên bố thông điệp dữ liệu chỉ có giá trị khi có thông báo xác nhận, thông điệp đó được xem là chưa gửi cho đến khi người khởi tạo nhận được xác nhận đã nhận được từ người nhận (Điều 16 khoản 2 điểm d); (4) Nếu người khởi tạo đã gửi thông điệp mà không tuyên bố yêu cầu xác nhận và chưa nhận được xác nhận, người khởi tạo có thể ấn định một khoảng thời gian hợp lý để người nhận xác nhận; nếu hết thời hạn đó mà vẫn không nhận được xác nhận thì có quyền xem như thông điệp đó chưa được gửi (Điều 16 khoản 2 điểm đ)
Top-k child IDs	D16_K2_Pd, D16_K2_Pđ, D17_K1, D16_K2_Pa, D14_K2_Pb
Retrieved Article IDs	D16, D17, D14
Evidence gap	thiếu Điều được dẫn chiếu; thiếu cấu trúc trong cùng Điều; có ứng viên dẫn chiếu ở bước duyệt tiếp theo
Candidate Article IDs	D37, D16

Trường	Nội dung truy vết
Graph-fetched Article IDs	D37, D16
Gate decision	accept D37, D16; reject D15, D18; không fail-open.
Draft answer rút gọn	Dựa vào ngữ cảnh cung cấp, tôi sẽ phân tích từng phần của câu hỏi: 1. Việc coi đơn hàng chưa xác lập: - Theo [Điểm đ, Khoản 2, Điều 16, Luật Giao dịch điện tử 2023], nếu bạn đã gửi thông điệp dữ liệu (đơn đặt hàng) mà không tuyên bố về việc người nhận phải gửi thông báo xác nhận và cũng chưa nhận được thông báo xác nhận, thì bạn có thể thông báo cho người nhận là chưa nhận được thông báo xác nhận. Nếu sau một khoảng thời gian hợp lý vẫn không nhận được phản hồi từ người nhận, bạn có quyền xem đơn đặt hàng đó như chưa được xác lập. 2. Thời điểm thông điệp dữ liệu được coi là đã 'nhận': - Theo [Khoản 1, Điều 17, Luật Giao dịch điện tử 2023], t...
Final answer rút gọn	Dựa vào ngữ cảnh và các tài liệu cung cấp, tôi có thể trả lời câu hỏi của bạn như sau: 1. Việc coi đơn đặt hàng chưa được xác lập: - Theo [Điểm d, Khoản 2, Điều 16, Luật Giao dịch điện tử 2023], nếu trước hoặc trong khi gửi thông điệp dữ liệu (trong trường hợp này là đơn đặt hàng), người khởi tạo đã tuyên bố rằng thông điệp dữ liệu chỉ có giá trị khi có thông báo xác nhận từ bên kia, thì đơn đặt hàng của bạn được coi là chưa gửi cho đến khi bạn nhận được thông báo xác nhận từ cửa hàng trực tuyến. Nếu không có thông báo xác nhận và sau một khoảng thời gian hợp lý mà bạn vẫn chưa nhận được phản hồi, bạn có quyền xem đơn đặt hàng đó chưa được x...

Trường	Nội dung truy vết
Required facts được bao phủ	Đối chiếu thủ công: câu cuối nêu rõ 2/4 fact (điểm a, d); điểm d chỉ được gộp một phần; liên kết Điều 37 → Điều 16 không được phát biểu rõ. Pipeline không ghi nhãn bao phủ theo fact.
Nhận xét	D37 được tìm từ cạnh THAM_CHIEU đi vào D16; D37 đồng thời có các cạnh đi ra D15, D16, D17, D18. Sinh lại; 6.316 token, 6 lượt gọi; không lỗi. Đoạn về chế tài trong câu cuối nằm ngoài required facts và cần xem là hạn chế.

Kịch bản 3 — Tham chiếu chéo Điều
Nhóm cross_reference - trace JSON thật

Câu hỏi testset cat2_07
Khi tôi gửi đơn đặt hàng điện tử cho một cửa hàng trực tuyến nhưng không thấy họ phản hồi xác nhận, tôi có thể coi như đơn hàng đó chưa được xác lập không? Và nói chung khi nào một thông điệp dữ liệu như đơn đặt hàng được xem là bên kia đã 'nhận' được về mặt pháp lý?

RETRIEVED
5 child / 3 Điều
lu_t_giao_d_ch_i_n_t_2023_D16 lu_t_giao_d_ch_i_n_t_2023_D17 lu_t_giao_d_ch_i_n_t_2023_D14

SỐ NHÓM GÁP GHI NHÃN
4
is_complete=False

Candidate → gate → graph fetch
CANDIDATE
lu_t_giao_d_ch_i_n_t_2023_D37 lu_t_giao_d_ch_i_n_t_2023_D16
GATE
accept accept reject reject
FETCHED
lu_t_giao_d_ch_i_n_t_2023_D37 lu_t_giao_d_ch_i_n_t_2023_D16

Kết quả
regenerated=true
Dựa vào ngữ cảnh và các tài liệu cung cấp, tôi có thể trả lời câu hỏi của bạn như sau:
1. **Việc coi đơn đặt hàng chưa được xác lập:**
- Theo [Điểm d, Khoản 2, Điều 16, Luật Giao dịch điện tử 2023], nếu trước hoặc trong khi gửi thông điệp dữ liệu (trong trường hợp này là đơn đặt hàng), người khởi tạo đã tuyên bố rằng thông điệp dữ liệu chỉ có giá trị khi có thông báo xác nhận từ bên kia, thì đơn đặt hàng của bạn được coi là chưa gửi cho đến khi bạn nhận được thông báo xác nhận từ cửa hàng trực tuyến. Nếu không có thông báo xác nhận và sau một khoảng thời gian hợp lý mà bạn vẫn chưa nhận được phản hồi, bạn có quyền xem đơn đặt hàng đó chưa được xác lập.

Nguồn: phiên chạy cục bộ ngày 04/08/2026 - Không chỉnh sửa quyết định retrieval/Critic.

Hình 4.9: Case study bổ sung Điều qua quan hệ THAM_CHIEU.

4.5.4. Ứng viên không liên quan bị semantic gate loại

Mục tiêu và đầu vào. Kiểm tra việc một ứng viên do cấu trúc graph gợi ý nhưng không liên quan trực tiếp bị loại trước khi vào ngữ cảnh cuối. Câu hỏi `cat1\_03` là: “Đăng tải thông tin sai sự thật, xuyên tạc, vu khống, xúc phạm danh dự nhân phẩm người khác trên trang thông tin điện tử thì bị xử phạt như thế nào?”

Trạng thái ban đầu và khoảng trống. Các Article ban đầu theo thứ hạng là D99, D4, D4a và D101. Critic phát hiện D99 và D4 chưa đủ cấu trúc, đồng thời graph gợi ý D106 từ một cạnh THAM_CHIEU xuất phát ở D4.

Quyết định gate và bằng chứng cuối. Gate từ chối D106 và D4; chỉ parent D99 được chấp nhận và xuất hiện trong `graph\_fetched\_dieu\_ids`. Vì vậy, bằng chứng cuối gồm top-k ban đầu cộng toàn văn D99, không chứa hai ứng viên bị loại. Trace không ghi fail-open.

Kết quả và kết luận. Câu trả lời sau sinh lại nêu mức phạt 20–30 triệu đồng, tịch thu tang vật/phương tiện, buộc gỡ đường dẫn sai sự thật và thu hồi/hoàn trả tên miền. Case này xác nhận gate có thể ngăn candidate cấu trúc làm phình context; nó không chứng minh tỷ lệ false-positive/false-negative của gate trên toàn bộ testset.

4.5.5. Gate exception và chính sách fail-open

Mục tiêu và thiết lập. Kiểm tra trực tiếp nhánh exception của `is\_candidate\_relevant` mà không gọi API trả phí và không thay đổi Qdrant. Phép thử dùng câu hỏi thật `cat1\_02` cùng parent D105 lấy read-only từ Qdrant Base (3.044 ký tự; SHA-256 `60377c48465dfa8e8007516c19849e80cdbbea807eb851ee2fef0dbc75c245b2`), sau đó thêm một `RuntimeError` có kiểm soát tại lời gọi LLM.

Hành vi quan sát. Hàm ghi cảnh báo “Relevance gate lỗi, mặc định coi là liên quan (fail-open)” và trả `True`. Theo workflow, candidate sẽ được xử lý như ứng viên được chấp nhận và có thể kích hoạt regeneration.

Kết luận và rủi ro. Kết quả `True` chỉ phản ánh chính sách duy trì recall khi lỗi kỹ thuật, không phải phán đoán semantic yes. Không tìm thấy trace production ghi nhận gate exception tự nhiên, nên khóa luận không báo cáo tần suất fail-open. Hệ thống vận hành thực tế cần log loại lỗi, tách `accept` khỏi `fail\_open`, đặt timeout/retry và cho phép chính sách fail-closed theo mức rủi ro.

4.5.6. Mixed-document safeguard sau khi mọi candidate bị từ chối

Mục tiêu và đầu vào. Kiểm tra safeguard khi gate không chấp nhận candidate nào nhưng top-k trộn nhiều văn bản. Câu `cat1\_02` hỏi về xử phạt khi thiết lập điểm cung cấp trò chơi điện tử công cộng không có giấy chứng nhận đủ điều kiện.

Trạng thái ban đầu và quyết định gate. Các Article ban đầu gồm D105, D103, D62 và D68 thuộc ít nhất hai văn bản. Critic tạo các candidate D5, D6, D105, D103 và D62; toàn bộ đều được ghi trong `rejected\_by\_relevance\_gate`, nên danh sách accepted sau gate rỗng.

Safeguard và bằng chứng cuối. Điều kiện nhiều văn bản được thỏa mãn, vì vậy code bổ sung parent đứng đầu D105 mà không chạy gate lần nữa, ghi `reinforced\_top\_dieu\_multi\_van\_ban=D105`, tạo `graph\_context` và sinh lại. Đây là đúng nhánh hậu kiểm trong mã nguồn, không phải candidate đã được semantic gate chấp nhận.

Kết quả và kết luận. Draft ghi sai mức phạt 1–3 triệu đồng và thiếu chế tài bổ sung. Final answer sau safeguard nêu 5–10 triệu đồng và đình chỉ hoạt động 01–03 tháng, khớp hai required facts của testset. Trace ghi 8 lượt gọi, 7.872 token toàn pipeline và 2.511 token ở lượt sinh cuối. Case cho thấy safeguard có thể hữu ích khi context ban đầu bị trộn, nhưng vì nó bỏ qua gate nên cần được đánh giá riêng thay vì mặc định xem là tối ưu.

4.6. Quy trình chạy đánh giá

run_evaluation.py đọc testset và lặp trên từng mode. Với mỗi record, script truyền query vào pipeline với skip_router=True, lưu answer, contexts, required facts, retrieved_dieu_ids_ranked, graph_fetched_dieu_ids, draft/critic report và hai nhóm token. Tùy chọn --resume tiếp tục batch; exception của từng câu được in rồi bỏ qua, do đó số dòng output phải được kiểm tra trước khi tổng hợp. score_evaluation.py tính retrieval metrics, Article metrics, LCR và RAGAS khi dependency/API sẵn sàng.

Bộ test hiện có 301 dòng và SHA-256 58e999c2a33f762e352ef6f477ea1ef49784b3cfa7c9b291ace24e354eba1b96. Tài liệu hướng dẫn cũ vẫn mô tả 50 câu; con số này không được dùng. Đầu ra chi tiết Base ngày 31/07/2026 hoàn chỉnh ở Naive và Expansion; artefact Critic đang lưu có schema evaluation nhưng chỉ bao phủ một tập con. Repository không có log đủ để xác định phạm vi thực thi của toàn bộ run Critic. Vì chưa có raw Critic đủ 301 câu và raw scoring artifacts tương ứng, số liệu tổng hợp ba chế độ được ghi là kết quả do tài liệu nguồn báo cáo; MRR/nDCG tái tính từ đầu ra chi tiết Naive được báo cáo riêng để không trộn hai phiên bản dữ liệu. Các tình huống ở Mục 4.5 không thay thế phép đánh giá đủ 301 câu.

4.7. Chấm Legal Completeness

Đối với LCR, mỗi lượt chấm chỉ nhận câu trả lời cần đánh giá và đúng một required fact; judge không nhận câu hỏi, reference answer, Article ID, context truy xuất hoặc tên phương pháp. Với mỗi fact, GPT-4o-mini ở temperature 0 được yêu cầu chỉ trả yes/no; parser hiện ánh xạ mọi output chứa chuỗi “yes” không phân biệt hoa thường thành 1 và các output khác thành 0. Prompt chấp nhận diễn đạt tương đương nhưng yêu cầu đúng con số và điều kiện pháp lý. Điểm câu là tỷ lệ fact đạt; LCR toàn bộ là macro-average trên 301 câu. Mã hiện hành không đặt rõ max-token, seed, timeout hay retry cho lượt LCR, vì vậy raw judge strings cần được lưu ở lần chạy sau để kiểm tra sai khác giữa quy tắc chứa chuỗi và parse yes/no chính xác.

RAGAS 0.1.22 được sử dụng để đánh giá bốn khía cạnh gồm Faithfulness, Answer Relevancy, Context Precision và Answer Correctness. Dữ liệu đầu vào của bộ đánh giá gồm câu hỏi, câu trả lời của hệ thống, các đoạn ngữ cảnh truy xuất và câu trả lời tham chiếu. Các giá trị không hợp lệ phát sinh trong quá trình chấm được loại theo từng chỉ số trước khi tổng hợp kết quả. Kết quả đánh giá được trình bày tại Mục 5.4.

Phương pháp vẫn có ba rủi ro. Thứ nhất, required facts và reference answer là silver standard. Thứ hai, judge có thể ưu tiên câu dài hoặc cách diễn đạt giống reference. Thứ ba, không có báo cáo agreement giữa judge và chuyên gia. Vì vậy,

LCR được dùng để so sánh nội bộ các cấu hình dưới cùng protocol, không phải chứng nhận độ đúng pháp lý tuyệt đối.

4.8. Kiểm thử phần mềm và kiểm soát lỗi

Các kiểm tra cần thực hiện gồm: unit test cho regex chunking và ID collision; kiểm tra tổng child/parent; kiểm tra hai collection Qdrant có payload tương thích; kiểm tra graph target tồn tại hoặc được đánh dấu placeholder; kiểm tra mapping Neo4j ID sang parent Qdrant; kiểm tra mode không lấy trùng Điều; kiểm tra token counter và checkpoint resume.

Một lỗi đáng chú ý là tổng số Khoản/Điểm trong Neo4j có thể thấp hơn Qdrant nếu extraction bỏ sót cạnh cấu trúc. `find_structurally_incomplete_dieu` vì vậy nhận `total_parts_override` từ nguồn đếm độc lập và dùng giá trị lớn hơn giữa hai nguồn. Thiết kế này tránh việc graph thiếu làm Critic kết luận sai rằng retrieval đã đủ.

4.9. Khả năng tái lập

Mức tái lập hiện tại là một phần. Kho mã nguồn hiện hành có quy trình xử lý, testset 301 dòng, `scripts/validate_testset.py` và đầu ra chi tiết Base Naive/Expansion; phép kiểm A1-A3 có thể tái chạy. Tuy nhiên, repository chưa lưu checkpoint/kết quả B1-B6, quyết định theo từng câu của bộ chấm LCR/RAGAS, raw output Critic đủ 301 câu, package lock hoàn chỉnh, thông số phần cứng, model digest, seed, latency và nhật ký API. Fine-tuned checkpoint vắng mặt; chỉ mục GTE sai kích thước. Do đó, một nhóm độc lập có thể kiểm tra corpus/chỉ mục và một phần retrieval, nhưng chưa thể tái tạo chính xác toàn bộ bảng LCR, RAGAS hoặc từng quyết định gate/judge. Việc thiếu artefact không được diễn giải thành kết luận về số câu mà thí nghiệm đầy đủ đã thực sự chạy.

5. KẾT QUẢ VÀ THẢO LUẬN

5.1. Độ tin cậy của bộ testset

Theo tài liệu phương pháp kiểm định testset, lớp A tìm thấy 0 lỗi `diu_id` và 0 lỗi `schema category` trên 301 record. Kiểm tra gần trùng gần cờ hai cặp: `cat4_new1_11` với `cat4_new3_03` có similarity 0,94; `cat4_new2_14` với `cat4_new3_04` có similarity 0,95. Tài liệu cho biết các cặp đã được rà soát thủ công và giữ lại, nhưng không kèm biên bản adjudication để kiểm tra tiêu chí giữ.

Lớp B được tài liệu phương pháp mô tả là chạy bằng o3-mini trên toàn văn các Điều gốc. Tỷ lệ đạt tổng là 99,2% cho required facts bám nguồn, 99,3% cho reference answer bám nguồn, và 100% cho bao phủ facts, độ tự nhiên/rõ ràng, nhu cầu dẫn nguồn và đúng category. Ba bản ghi bị gần cờ ở B1 là `cat3_04`, `cat1_new3_04`, `cat4_new2_03`; trong đó `cat3_04` và `cat4_new2_03` cũng bị gần cờ ở B2. Gần cờ không tự động có nghĩa cả bản ghi sai: cần đọc chi tiết quyết định của bộ chấm để xác định fact hoặc mệnh đề cụ thể, nhưng tập chi tiết chưa có trong dự án.

Bảng 5.1. Kết quả kiểm định testset do tài liệu phương pháp báo cáo

Lớp/tiêu chí	Chế tài kép	Tham chiếu chéo	Đa thành phần	Đối chứng	Toàn bộ
A1 — ID tồn tại	0 lỗi	0 lỗi	0 lỗi	0 lỗi	0 lỗi
A2 — Schema category	0 lỗi	0 lỗi	0 lỗi	0 lỗi	0 lỗi
A3 — Cặp gần trùng	—	—	—	2 cặp	2 cặp
B1 — Facts grounded	99,3%	100,0%	99,2%	99,0%	99,2 %
B2 — Answer grounded	100,0%	100,0%	99,2%	99,0%	99,3 %

Lớp/tiêu chí	Chế tài kép	Tham chiếu chéo	Đa thành phần	Đối chứng	Toàn bộ
B3 — Facts được đáp án bao phủ	100,0%	100,0%	100,0%	100,0 %	100,0 %
B4 — Tự nhiên, rõ ràng	100,0%	100,0%	100,0%	100,0 %	100,0 %
B5 — Cần dẫn nguồn	100,0%	100,0%	100,0%	100,0 %	100,0 %
B6 — Đúng category	100,0%	100,0%	100,0%	100,0 %	100,0 %

Các kết quả trên cải thiện mức minh bạch so với việc chỉ gọi testset là silver standard: đã có script, định nghĩa tiêu chí, đối chiếu toàn văn và danh sách trường hợp cần rà soát. Tuy nhiên, chúng chưa thay thế kiểm định độc lập. Không có checkpoint, kết quả đầu ra theo từng câu, phiên bản mô hình của lần chấm hoặc các tệp tổng hợp/chi tiết/lỗi trong dự án; cũng chưa có chuyên gia luật đánh giá mù và đo agreement. Vì vậy, khóa luận dùng testset cho so sánh nội bộ nhưng không diễn giải các tỷ lệ 99–100% như chứng nhận “đúng pháp lý tuyệt đối”.

5.2. Kết quả truy xuất ban đầu của Base retriever

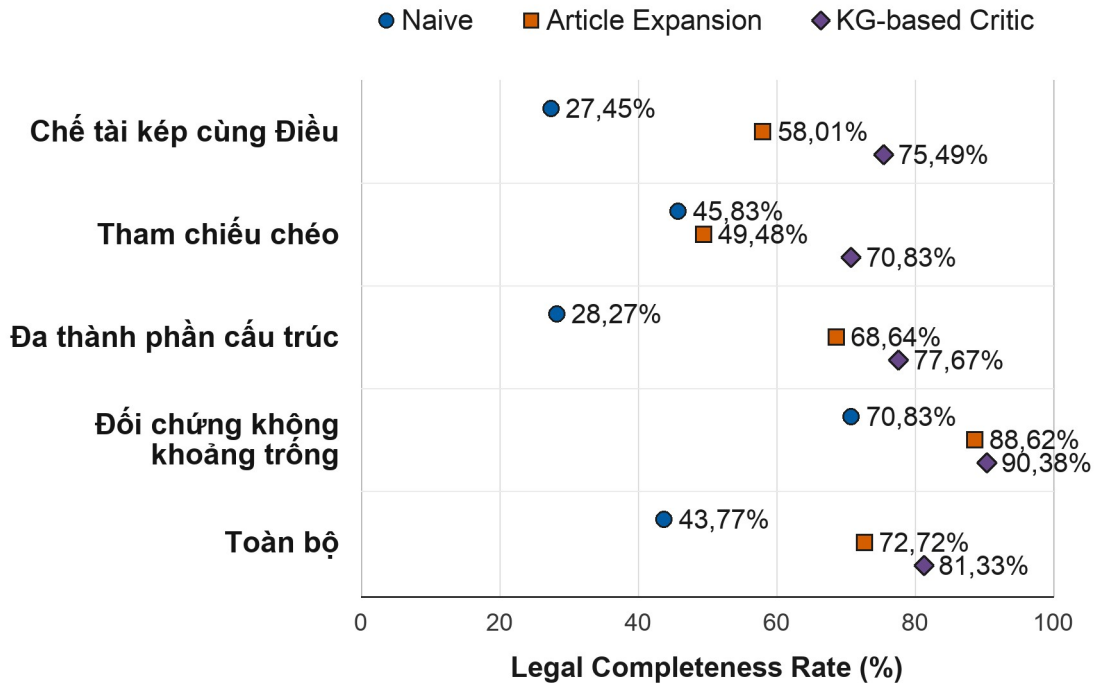
Bảng 5.2. Chỉ số truy xuất Base theo hai phiên bản dữ liệu bằng chứng

Chỉ số	Tái tính từ đầu ra chi tiết Naive 301 câu	Bộ kết quả thực nghiệm tổng hợp	Trạng thái
MRR@5	0,9028	0,8560	Hai phiên bản dữ liệu khác nhau

Chỉ số	Tái tính từ đầu ra chi tiết Naive 301 câu	Bộ kết quả thực nghiệm tổng hợp	Trạng thái
nDCG@5	0,9122	0,8660	Hai phiên bản dữ liệu khác nhau
Article precision	0,3095	0,3080	Gần nhau, không tự động coi là cùng lần chạy
Article recall	0,9585	0,9520	Gần nhau, không tự động coi là cùng lần chạy

Cột thứ nhất được tái tính trực tiếp từ 301 dòng đầu ra chi tiết Base Naive có mã băm 0a5e...5893; cột thứ hai là số liệu trong bộ kết quả thực nghiệm tổng hợp. Vì MRR và nDCG chênh lệch đáng kể, khóa luận không âm thầm chọn một giá trị. Kết quả chương này dùng bộ số liệu tổng hợp khi cần so sánh đồng bộ với LCR/RAGAS của ba chế độ, còn cột tái tính cho thấy tệp đầu ra hiện tại thuộc phiên bản dữ liệu khác. Không có kết luận về fine-tuning hoặc GTE trong phần kết quả chính.

5.3. Legal Completeness Rate



Hình 5.1: Legal Completeness Rate của ba chế độ dùng chung Base retriever

Bảng 5.3. Legal Completeness Rate trên 301 câu hỏi

Nhóm câu hỏi	Số câu	Naive	Article Expansion	KG-based Critic
Chế tài kép cùng Điều	51	27,45%	58,01%	75,49%
Tham chiếu chéo	16	45,83%	49,48%	70,83%
Đa thành phần cấu trúc	130	28,27%	68,64%	77,67%
Đối chứng không khoảng trống	104	70,83%	88,62%	90,38%
Toàn bộ	301	43,77%	72,72%	81,33%

Theo bộ kết quả thực nghiệm tổng hợp, Article Expansion cao hơn Naive 28,95 điểm phần trăm trên toàn bộ testset; Critic cao hơn Expansion 8,61 điểm và cao hơn Naive 37,56 điểm. Nhóm tham chiếu chéo tăng từ 49,48% ở Expansion lên 70,83%

ở Critic, chênh lệch 21,35 điểm và phù hợp với mục đích của THAM_CHIEU. Nhóm chế tài kép và đa thành phần cũng cải thiện khi khôi phục parent Article. Tuy nhiên, đây là liên hệ mô tả giữa kiến trúc và chỉ số, không phải bằng chứng nhân quả, vì chưa có ablation, repeated runs hoặc paired test. Nhóm no-gap chỉ tăng 1,76 điểm từ Expansion lên Critic; do repository chưa lưu raw output Critic và judge decisions đủ 301 câu, chưa xác định được bao nhiêu trường hợp gate/regeneration làm tốt hơn hoặc làm xấu đi một câu vốn đã đủ ngữ cảnh.

5.4. Kết quả RAGAS

Bảng 5.4. Kết quả RAGAS của cấu hình Base

Chỉ số	Naive	Article Expansion	KG-based Critic
Faithfulness	0,705	0,841	0,857
Answer Relevancy	0,349	0,597	0,593
Context Precision	0,965	0,963	0,961
Answer Correctness	0,544	0,657	0,668

RAGAS đánh giá hệ thống trên bốn khía cạnh gồm mức độ trung thực của câu trả lời đối với ngữ cảnh, mức độ liên quan của câu trả lời, độ tập trung của ngữ cảnh truy xuất và mức độ phù hợp với câu trả lời tham chiếu. Kết quả tại Bảng 5.4 cho thấy Article Expansion và KG-based Critic đều có Faithfulness và Answer Correctness cao hơn Naive RAG. KG-based Critic đạt Faithfulness 0,857 và Answer Correctness 0,668, là các giá trị cao nhất trong ba chế độ. Answer Relevancy của Article Expansion và KG-based Critic lần lượt đạt 0,597 và 0,593, cho thấy mức chênh lệch nhỏ giữa hai phương pháp. Context Precision giảm nhẹ từ 0,965 ở Naive RAG xuống 0,961 ở KG-based Critic. Nhìn chung, các kết quả mô tả cho thấy hai chế độ bổ sung cấu trúc đi kèm mức cải thiện về tính trung thực và độ đúng của câu trả lời, trong khi độ tập trung của ngữ cảnh vẫn được duy trì ở mức tương đối ổn định.

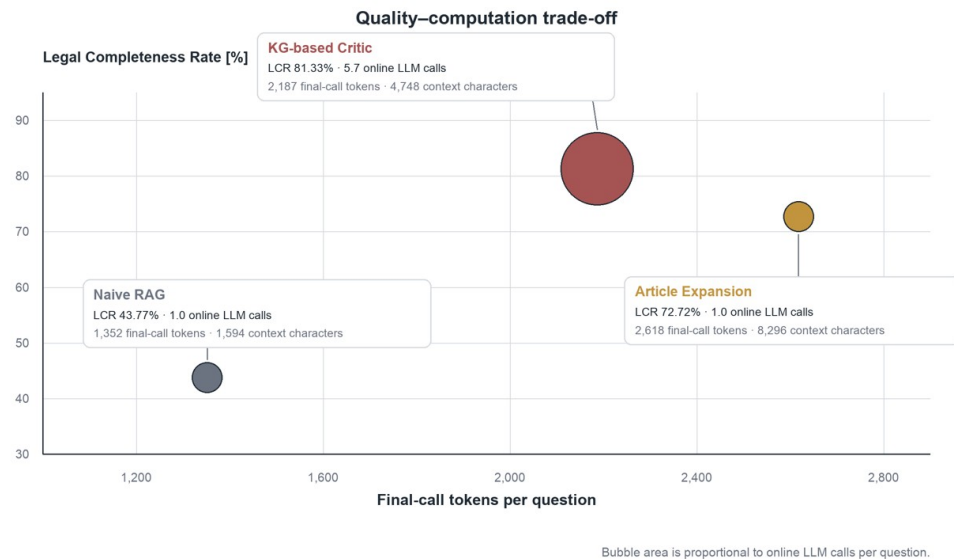
5.5. Precision, recall và độ dài ngữ cảnh theo Điều

Bảng 5.5. Chỉ số context ở cấp Điều

Chỉ số	Naive	Article Expansion	KG-based Critic
Article precision	0,308	0,303	0,308
Article recall	0,952	0,952	0,962
Context characters	1.594	8.296	4.748

Theo bộ kết quả thực nghiệm tổng hợp, Critic có Article recall cao nhất 0,962, trong khi precision bằng Naive và cao hơn nhẹ Expansion. Diễn giải thận trọng là ngữ cảnh cuối của Critic chứa thêm Điều đúng mà số liệu tổng hợp không cho thấy giảm precision; chưa thể khẳng định ý nghĩa thống kê. Article Expansion tạo ngữ cảnh dài nhất 8.296 ký tự. Critic dùng 4.748 ký tự, thấp hơn 42,8% so với Expansion nhưng vẫn cao hơn Naive. Đây là điểm cân bằng mô tả giữa độ phủ và lượng nội dung, không phải tối ưu đã được chứng minh.

5.6. Chi phí token và số lượt gọi mô hình



Hình 5.2: Đánh đổi giữa LCR, token của lần sinh cuối, số ký tự ngữ cảnh và số lượt gọi mô hình online

Bảng 5.6. Chi số chi phí online trung bình trên mỗi câu hỏi

Chỉ số	Naive	Article Expansion	KG-based Critic
Final-call tokens/câu	1.352	2.618	2.187
Online LLM calls/câu	1,0	1,0	5,7

KG-based Critic thực hiện trung bình 5,7 lượt gọi mô hình online cho mỗi câu, trong khi Naive và Article Expansion đều dùng 1,0 lượt. Đối lại, lượt sinh câu trả lời cuối của Critic xử lý 2.187 token và 4.748 ký tự ngữ cảnh, thấp hơn 2.618 token và 8.296 ký tự của Expansion, tương ứng giảm 16,5% và 42,8%. Điều này không có nghĩa Critic rẻ hơn toàn pipeline vì còn các lượt sinh nháp và gate. Tổng pipeline token 1.516/2.782/6.540 của nguồn cũ bị loại khỏi bảng chính do ranh giới đếm không đồng nhất: baseline cũ chứa router, còn benchmark hiện hành bỏ router. Hình 5.2 và Bảng 5.6 vì vậy chỉ dùng các đại lượng có định nghĩa thống nhất trong bài báo cuối.

5.7. Trả lời các câu hỏi nghiên cứu

5.7.1. RQ1 - Khác biệt giữa ba mode

Trên Base retriever, Naive có LCR 43,77%; Article Expansion đạt 72,72%; Critic đạt 81,33%. Nhóm tham chiếu chéo tăng 21,35 điểm từ Expansion lên Critic. Các kết quả phù hợp với giả thuyết rằng top-5 child thiếu cấu trúc và graph có thể gợi ý Điều được dẫn chiếu. Vì số liệu là aggregate do nguồn báo cáo và không có ablation, kết luận dừng ở mức liên hệ mô tả, không khẳng định quan hệ nhân quả.

5.7.2. RQ2 - Ảnh hưởng của khôi phục và sửa bằng chứng

Article Expansion giữ Article recall ở 0,952 nhưng tăng context từ 1.594 lên 8.296 ký tự; Critic nâng recall lên 0,962 với context 4.748 ký tự. Faithfulness tăng từ 0,705 lên 0,841 và 0,857; Answer Correctness tăng từ 0,544 lên 0,657 và 0,668. Context Precision giảm nhẹ. Kết quả cho thấy bổ sung cấu trúc có liên hệ với độ phủ và độ trung thực cao hơn, nhưng chưa chứng minh từng thành phần graph hoặc gate là nguyên nhân vì thiếu ablation.

5.7.3. RQ3 - Đánh đổi chất lượng và chi phí

Critic đổi thêm các lượt gọi trung gian để lấy LCR và Article recall cao hơn: online calls tăng từ 1,0 ở Expansion lên 5,7. Tuy nhiên, final-call tokens giảm từ 2.618 xuống 2.187 và context giảm từ 8.296 xuống 4.748 ký tự. Vì vậy, Naive phù hợp luồng tối giản, Expansion phù hợp bài toán chủ yếu thiếu phần trong cùng Điều với một lần sinh, còn Critic phù hợp hơn khi ưu tiên tính đầy đủ hoặc cross-reference và chấp nhận nhiều lượt gọi hơn. Tổng pipeline token không được dùng để xếp hạng chi phí khi hai nguồn đo khác ranh giới router.

5.8. Phân tích lỗi và rủi ro

Retriever có thể đưa Điều nhiều vào top-k. Nếu Critic kiểm tra mọi Điều, mỗi Điều nhiều có thể kéo theo tham chiếu riêng và làm context phình to. Ngưỡng tương đối và giới hạn bốn Điều là cơ chế giảm rủi ro này. Tuy nhiên, ngưỡng cố định có thể bỏ sót câu hỏi thực sự cần nhiều Điều điểm thấp hơn; cần ablation để chọn giá trị.

Graph extraction có thể bỏ entity, sai relation hoặc tạo placeholder không thể resolve. Đặc biệt, một member extraction bị trùng/mislabeled cho thấy kiểm soát tệp bằng chứng chưa đủ. Critic có override số phần từ Qdrant, nhưng THAM_CHIEU vẫn phụ thuộc graph. Một graph sai có thể bỏ sót Điều cần lấy hoặc lấy Điều không liên quan.

Semantic gate có thể false negative và loại bằng chứng cần thiết, hoặc false positive và chấp nhận context nhiều. Cấu hình fail-open bảo vệ recall nhưng làm tăng rủi ro nhiều khi model/API lỗi. Production nên phân biệt timeout, parse error và quyết định yes; exception hiện hành không được xem như bằng chứng liên quan đã được model xác nhận.

LLM judge có thể thiên lệch theo độ dài. Critic thường tạo answer/context khác độ dài so với các mode khác, nên bias độ dài có thể ảnh hưởng LCR. Cần đánh giá mù mode, đảo thứ tự, lấy mẫu chuyên gia và đo agreement để kiểm tra.

5.9. Giới hạn nghiên cứu

Thứ nhất, kết quả LCR, RAGAS và chi phí hiện có là số liệu tổng hợp trên 301 câu do tài liệu nguồn báo cáo. Repository chưa lưu đầy đủ raw output Critic 301 câu, quyết định judge, repeated runs, confidence interval hoặc dữ liệu cần thiết cho paired statistical test. Mọi so sánh là mô tả, không phải bằng chứng về ý nghĩa thống kê; số record của artefact Critic cục bộ không cho phép suy ra phạm vi thực thi của toàn bộ thí nghiệm.

Thứ hai, kho mã nguồn hiện hành có script kiểm định testset và phép kiểm A1-A3 đã được tái chạy, nhưng không có checkpoint, kết quả B1-B6 hoặc biên bản rà soát hai cặp gần trùng. Quy trình xây dựng câu hỏi, vai trò người chú giải, trình độ pháp lý, guideline, inter-annotator agreement và adjudication cũng chưa được tài liệu hóa đầy đủ. Required facts và reference answer vì vậy vẫn là silver standard có kết quả tự động được tài liệu nguồn báo cáo, chưa phải gold standard chuyên gia.

Thứ ba, LCR chưa được kiểm định với luật sư; quyết định theo từng fact và nhật ký lỗi của bộ chấm chưa được cung cấp đầy đủ. RAGAS phụ thuộc phiên bản và bộ đánh giá, nên cần lưu prompt và kết quả đầu ra của từng câu.

Thứ tư, corpus chỉ bao phủ một số lĩnh vực và năm PDF chưa vào index. Phép chạy chunker và index hiện hành xác nhận 1.211 parent; manifest 1.209 là tài liệu cũ mâu thuẫn. Văn bản còn thay đổi hiệu lực, do đó kết quả phản ánh snapshot thí nghiệm, không mặc định là luật hiện hành tại mọi thời điểm.

Thứ năm, chưa có đánh giá độc lập chất lượng KG. Tập trích xuất có một trường hợp trùng hoặc gán nhãn sai; lần kiểm tra trực tiếp xác nhận 15.666 node, 14.137 quan hệ và 1.612 quan hệ THAM_CHIEU, nhưng chưa có precision/recall được đánh giá độc lập ở cấp quan hệ. Không thể tách đóng góp của kiểm tra trong cùng Điều, duyệt đồ thị, cổng liên quan và sinh lại vì chưa có component ablation.

Thứ sáu, chưa có môi trường thực nghiệm khóa phiên bản hoàn chỉnh: hardware, OS/CUDA, package lock, model digest, seed, latency, energy và monetary cost. Fine-tuned checkpoint không tồn tại; GTE index hiện tại sai kích thước so với model card. Các metric token cũng thiếu định nghĩa chi tiết về cached/system/tool token.

5.10. Hàm ý thiết kế

Kết quả gợi ý không nên coi “nhiều context hơn” là chiến lược mặc định. Parent expansion giải quyết tốt thiếu cấu trúc cùng Điều nhưng tăng mạnh độ dài. Graph critique có thể dùng như lớp kiểm tra sau retrieval, đặc biệt cho domain có quan hệ dẫn chiếu rõ. Tuy nhiên, để đáng tin cậy, graph cần provenance, version và đánh giá riêng; gate cần log và chính sách lỗi rõ ràng.

Một kiến trúc production có thể dùng routing theo rủi ro. Câu hỏi đơn giản chạy naive; câu hỏi có nhiều điều kiện cùng Điều chạy expansion; câu hỏi có dấu hiệu dẫn chiếu hoặc chế tài chạy critic. Routing này phải được đánh giá trên dữ liệu giữ lại, không thiết kế từ chính testset 301 câu.

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Khóa luận đã trình bày một hệ thống hỏi đáp pháp luật tiếng Việt kết hợp chunking Điều - Khoản - Điểm, hybrid retrieval, parent Article reconstruction, Knowledge Graph và Critic Agent. Điểm cốt lõi là thay đổi vai trò của graph: graph không trực tiếp thay văn bản để trả lời, mà phát hiện vị trí có khả năng thiếu và hướng dẫn hệ thống lấy thêm parent Article. Semantic gate tiếp tục lọc ứng viên, còn regeneration chỉ xảy ra khi có bằng chứng được chấp nhận.

Trên số liệu tổng hợp của 301 câu hỏi dùng Base retriever, bài báo cuối báo cáo LCR 43,77% cho Naive, 72,72% cho Article Expansion và 81,33% cho Critic. Mức cải thiện của Critic so với Expansion lớn hơn ở nhóm chế tài kép cùng Điều (17,48 điểm phần trăm) và tham chiếu chéo (21,35 điểm), phù hợp với mục tiêu sửa bằng chứng cấu trúc nhưng chưa chứng minh quan hệ nhân quả do thiếu ablation. Critic dùng trung bình 5,7 lượt gọi online, cao hơn 1,0 của Expansion; đồng thời lượt sinh cuối dùng ít hơn 16,5% token và 42,8% ký tự ngữ cảnh. Tài liệu kiểm định testset báo cáo A1–A2 không có lỗi, B1–B2 đạt 99,2–99,3% và B3–B6 đạt 100%; khóa luận nêu rõ chưa có kết quả B1–B6 theo từng câu hoặc xác nhận chuyên gia để tái kiểm. Do repository chưa lưu đầy đủ raw output Critic 301 câu và raw judge/checkpoint cần thiết để tái tính độc lập, LCR/RAGAS tiếp tục được ghi là số liệu tổng hợp `REPORTED`; số record của artefact cục bộ không được dùng để kết luận toàn bộ Critic chỉ chạy từng ấy câu. Fine-tuned và GTE bị loại khỏi trực chính vì thiếu checkpoint hoặc chỉ mục không hợp lệ.

Các kết luận được giới hạn ở phạm vi snapshot, protocol và metric đã có. Khóa luận không khẳng định hệ thống thay thế chuyên gia luật, không suy luận ý nghĩa thống kê từ aggregate, và không xem LLM judge là thẩm định pháp lý.

6.2. Hướng phát triển

Hướng ưu tiên thứ nhất là hoàn thiện reproducibility package: phát hành đầy đủ raw artifacts theo từng câu cho cả ba mode Base trên bộ 301 câu, bao gồm ranked

lists, context, draft, critic trace, gate decision, prompt, judge decision và log token; lưu checkpoint cùng summary/details/issues của kiểm định testset; khóa phiên bản dependency, model digest/revision, corpus và từng index bằng hash.

Hướng thứ hai là xây dựng gold standard có chuyên gia. Cần guideline, ít nhất hai người chú giải độc lập, measurement agreement và adjudication. Một mẫu nên được double-blind so sánh với GPT-4o-mini judge để ước lượng false positive/negative của LCR.

Hướng thứ ba là sửa và đánh giá KG. Archive cần thay file trùng/mislabeled, resolve placeholder, xuất graph immutable và đo precision/recall trên mẫu node/relation. Ablation riêng cho compound-penalty check, structural check, THAM_CHIEU, gate và regeneration sẽ xác định đóng góp của từng thành phần.

Hướng thứ tư là quản lý thời gian hiệu lực. Mỗi chunk nên có effective_from, effective_to, supersedes, nguồn chính thức và ngày tải. Retrieval cần lọc theo thời điểm người dùng hỏi. Khi văn bản được sửa đổi, pipeline phải cập nhật Qdrant và Neo4j trong cùng transaction/version.

Hướng thứ năm là tối ưu chi phí. Có thể dùng model gate nhỏ hơn, batch candidate, cache graph query, hoặc router rủi ro để chỉ gọi Critic khi câu hỏi có khả năng thiếu cấu trúc. Cần báo cáo latency và monetary cost song song với token.

Hướng cuối là phát triển API và giao diện có khả năng giải thích. Câu trả lời nên hiển thị Điều/Khoản/Điểm, văn bản, ngày hiệu lực, context đã dùng và cảnh báo khi graph/gate không chắc chắn. Người dùng phải có đường dẫn tới văn bản gốc và cơ chế phản hồi để phát hiện lỗi.

TÀI LIỆU THAM KHẢO

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020.
- [2] P. M. Kien, H.-T. Nguyen, N. X. Bach, V. Tran, M. L. Nguyen, and T. M. Phuong, “Answering legal questions by learning neural attentive text representation,” in *Proc. 28th Int. Conf. Computational Linguistics*, pp. 988-998, 2020, doi: 10.18653/v1/2020.coling-main.86.
- [3] S. Khazaeli, J. Punuru, C. Morris, S. Sharma, B. Staub, M. Cole, S. Chiu-Webster, and D. Sakalloy, “A free format legal question answering system,” in *Proc. Natural Legal Language Processing Workshop*, pp. 107-113, 2021, doi: 10.18653/v1/2021.nllp-1.11.
- [4] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157-173, 2024, doi: 10.1162/tacl_a_00638.
- [5] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbd1-Abstract-Conference.html
- [6] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008, doi: 10.1017/CBO9780511809071.
- [7] X. Zhang *et al.*, “mGTE: Generalized long-context text representation and reranking models for multilingual text retrieval,” in *Proc. EMNLP 2024*:

- Industry Track*, pp. 1393-1412, 2024, doi: 10.18653/v1/2024.emnlp-industry.103.
- [8] N. Guha *et al.*, “LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 36, Datasets and Benchmarks Track, 2023.
- [9] Z. Fei, X. Shen, D. Zhu, F. Zhou, Z. Han, A. Huang, S. Zhang, K. Chen, Z. Yin, Z. Shen, J. Ge, and V. Ng, “LawBench: Benchmarking legal knowledge of large language models,” in *Proc. EMNLP*, pp. 7933-7962, 2024, doi: 10.18653/v1/2024.emnlp-main.452.
- [10] J. Lee, D. Kim, S. Hwang, H. Kim, and G. Lee, “KoBLEX: Open legal question answering with multi-hop reasoning,” in *Proc. EMNLP*, pp. 4019-4053, 2025, doi: 10.18653/v1/2025.emnlp-main.200.
- [11] A. B. Hou, O. Weller, G. Qin, E. Yang, D. Lawrie, N. Holzenberger, A. Blair-Stanek, and B. Van Durme, “CLERC: A dataset for U.S. legal case retrieval and retrieval-augmented analysis generation,” in *Findings of NAACL*, pp. 7913-7928, 2025, doi: 10.18653/v1/2025.findings-naacl.441.
- [12] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, “RAGAs: Automated evaluation of retrieval augmented generation,” in *Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations*, pp. 150-158, 2024, doi: 10.18653/v1/2024.eacl-demo.16.
- [13] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, “ARES: An automated evaluation framework for retrieval-augmented generation systems,” in *Proc. NAACL*, pp. 338-354, 2024, doi: 10.18653/v1/2024.naacl-long.20.
- [14] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, vol. 36, Datasets and Benchmarks Track, 2023.

- [15] Alibaba-NLP, “gte-multilingual-base,” *Hugging Face*, 2024. [Online]. Available: <https://huggingface.co/Alibaba-NLP/gte-multilingual-base>. [Accessed: Jul. 21, 2026].
- [16] AITeamVN, “Vietnamese_Embedding_v2,” *Hugging Face*, 2025. [Online]. Available: https://huggingface.co/AITeamVN/Vietnamese_Embedding_v2. [Accessed: Aug. 5, 2026].